Хамгаалалтын гарын авлага

Лаб 1–5, Бие даалт 1–2

Алгоритмын зохиомж ба шинжилгээ (F.CSM301)

Юу асууж байна, хэрхэн бодсон, код юу хийдэг, яагаад

Б. Билгүүн (**B190910014**)

Улаанбаатар, 2026

Агуулга

1 Энэ гарын авлагыг хэрхэн ашиглах вэ **3**

2 Бүх лабад хэрэгтэй үндсэн ойлголт **3**

2.1 Алгоритмын хугацааг яаж хэмждэг вэ . . . 3
2.2 Санах ой . . . 3
2.3 Дахин дуудах (рекурс) ба тэгшитгэл . . . 4
2.4 Алгоритм зохиох гурван гол арга . . . 4
2.5 Хэмжилт ба тест . . . 5
2.6 Ажлын урсгал (Git, GitLab) . . . 5

3 Лаб 1 — Орчин бэлтгэх, Фибоначчи, анаграмм **5**

3.1 Энэ лаб юуны тухай вэ . . . 5
3.2 Орчин (тайлангийн 1–3-р хэсэг) . . . 5
3.3 Фибоначчийн тоо . . . 6
3.4 Анаграмм . . . 7
3.5 Багшийн асууж болох асуулт . . . 7

4 Лаб 2 — Ажиллах хугацааг тодорхойлох **7**

4.1 Энэ лаб юуны тухай вэ . . . 8
4.2 Даалгавар 1 — Зөөх эрэмбэлэлт (insertion sort) . . . 8
4.3 Даалгавар 2 — Нэгтгэн эрэмбэлэлт (merge sort) . . . 8
4.4 Даалгавар 3 — Хоёртын хайлт . . . 9
4.5 Даалгавар 4 — Хоёртын хайлттай зөөх эрэмбэлэлт . . . 10
4.6 Даалгавар 5 — Дарааллын зөрөө (ranking distance) . . . 10
4.7 Онлайн бодлого . . . 11
4.8 Багшийн асууж болох асуулт . . . 11

5 Лаб 3 — Хэсэгчлэх арга **11**

5.1 Энэ лаб юуны тухай вэ . . . 11
5.2 Ерөнхий арга — тохиолдлыг яаж тодорхойлох вэ . . . 11
5.3 Даалгавар 1 — Давталттай матриц үржүүлэх . . . 12
5.4 Даалгавар 2 — Дахин дуудах матриц үржүүлэх . . . 13
5.5 Даалгавар 4 (B) — Штрассены алгоритм . . . 13
5.6 Даалгавар 3 ба 5 — Матриц–вектор, Гауссын арга . . . 14
5.7 Уншлага (в) — Фибоначчи матрицын зэргээр (A) . . . 14
5.8 A төвшин — Гауссын сайжруулалт ба Якоби . . . 15
5.9 Онлайн бодлого . . . 15
5.10 Багшийн асууж болох асуулт . . . 16

6 Лаб 4 — Динамик программчлал **16**

6.1 Энэ лаб юуны тухай вэ . . . 16
6.2 Даалгавар 1 — Ган саваа (rod cutting) . . . 16

6.3 Шинжилгээ . . . 17
6.4 Даалгавар 3 — LCS (хамгийн урт ерөнхий дэд дараалал) . . . 17
6.5 Даалгавар 4 (B) — Матрицуудын цуваа үржвэр . . . 18
6.6 Даалгавар 5 (B) — Хоёр файлын ялгаа (diff) . . . 18
6.7 A төвшин — сайжруулалт . . . 18
6.8 Онлайн бодлого . . . 18
6.9 Багшийн асууж болох асуулт . . . 19

7 Лаб 5 — Хомхойлох алгоритм 19
7.1 Энэ лаб юуны тухай вэ . . . 19
7.2 Даалгавар 1 — Ажиллагааг түүвэрлэх (activity selection) . . . 19
7.3 Даалгавар 2 — Хэсэгчилж авч болох үүргэвч (fractional knapsack) . . . 20
7.4 Даалгавар 3 — Хариулт мөнгө ба min_coins . . . 20
7.5 Уншлага — каноник систем . . . 20
7.6 Даалгавар 4 (B) — Хаффманы код . . . 21
7.7 Даалгавар 5 (B) — Оновчтой кэш (furthest-in-future) . . . 21
7.8 A төвшин — сайжруулалт . . . 22
7.9 Онлайн бодлого . . . 22
7.10 Багшийн асууж болох асуулт . . . 22

8 Бие даалт — хамгаалалтын бэлтгэл 23
8.1 Хамгаалалт хэрхэн явагдах вэ . . . 23
8.2 Бие даалт 1 — Хэсэгчлэх ба дахин дуудах арга (`prj1/B190910014`) . . . 23
8.3 Бие даалт 2 — Графын хайлт, мөрийн хайлт, хомхойлох (`prj2/B190910014`) . . . 27
8.4 Хамгаалалтын өмнөх шалгах жагсаалт . . . 30

9 Нэгдсэн зураг — бүх зүйл хэрхэн холбогдох вэ 31

10 Түгээмэл асуултын хурдан хариулт 32

1 Энэ гарын авлагыг хэрхэн ашиглах вэ

Энэ баримт нь **лаб, бие даалтаа өөрөө хамгаалахад** зориулагдсан. Лаб бүрийн хувьд:

1. **Юуны тухай вэ** — том зураг, энгийн үгээр.
2. **Даалгавар бүр** — юу асууж байна, ямар санаагаар бодсон, код мөр мөрөөрөө юу хийдэг.
3. **Яагаад ийм хугацаа, санах ой вэ** — тоог хаанаас гаргасан.
4. **Багшийн асууж болох асуулт ба хариулт** — чангаар, өөрийн үгээр хэлж дасгал хийгээрэй.

Зөвлөгөө:

- Код бүрийг өөрийн компьютер дээр нээж, энэ баримттай зэрэгцүүлэн уншаарай.
- Тест ажиллуулж үзээрэй: `cd labNN/B190910014/python && .venv/bin/pytest -v`, `cd java && mvn test`.
- Хариултаа цээжлэх биш, **ойлгох** нь чухал. «Яагаад?» гэж асуувал санаагаа өөрийн үгээр тайлбарлах хэрэгтэй.
- Багш ихэвчлэн «энэ мөр юу хийдэг вэ?», «энэ тоо хаанаас гарсан бэ?», «ийм өөрчилбөл юу болох вэ?» гэж асуудаг.

2 Бүх лабад хэрэгтэй үндсэн ойлголт

2.1 Алгоритмын хугацааг яаж хэмждэг вэ

Алгоритмын хурдыг **секундээр биш, алхмын тоогоор** хэмждэг. Секунд нь компьютер, хэлнээс хамаардаг, харин алхмын тоо нь оролтын хэмжээ **n**-ээс л хамаарна.

Жишээ: n урттай массивын бүх гишүүнийг нэг удаа харвал n алхам. Хос бүрийг харвал $n\cdot(n-1)/2 \approx n^2/2$ алхам.

Өсөлтийн эрэмбэ. n томрох үед хамгийн хурдан өсөх гишүүн л чухал. $3n^2 + 100n + 7$ нь n их үед бараг $3n^2$ шиг өсөх тул «n^2-ын эрэмбэтэй» гэнэ. Тогтмол үржүүлэгч (3) ба бага гишүүдийг $(100n + 7)$ орхино.

Тэмдэглэгээ	Утга	Энгийнээр
$f(n) = O(g(n))$	дээрээс хязгаарлагдсан	«хамгийн ихдээ g шиг хурдан өснө»
$f(n) = \Omega(g(n))$	доороос хязгаарлагдсан	«хамгийн багадаа g шиг өснө»
$f(n) = \Theta(g(n))$	хоёр талаас	«яг g шиг өснө»

Θ-ийн тодорхойлолт: $c_1 > 0$, $c_2 > 0$, n_0 тогтмолууд олдоод $n \geq n_0$ бүрд $c_1\cdot g(n) \leq f(n) \leq c_2\cdot g(n)$ биелнэ. Лаб 2-т $n(n-1)/2 = \Theta(n^2)$-ийг $c_1 = 1/4$, $c_2 = 1/2$, $n_0 = 2$ гэж баталсан.

Түгээмэл эрэмбүүд (удаашралаар нь өсөхөөр): $1 < \lg n < n < n \lg n < n^2 < n^3 < 2^n < n!$

n	lg n	n lg n	n^2	2^n
10	3.3	33	100	1 024
1 000	10	10 000	1 000 000	хэт их
10^6	20	$2\cdot10^7$	10^{12}	хэт их

Компьютер секундэд $\approx 10^8$–10^9 энгийн үйлдэл хийдэг (Python $\approx 10^7$). Тиймээс $n = 10^6$ үед n^2 алгоритм ажиллах боломжгүй, $n \lg n$ хангалттай.

2.2 Санах ой

- **Нэмэлт санах ой:** оролтоос гадна алгоритмд хэрэгтэй санах ой (түр массив, хүснэгт, стек).
- **Дахин дуудалтын стек:** функц өөрийгөө дуудах бүрд стект нэг «давхарга» нэмэгдэнэ. Гүн нь d бол $\Theta(d)$ санах ой.

- **Хэмжилт:** Python-д `tracemalloc` нь **оргил** санах ойг (нэгэн зэрэг эзэлсэн хамгийн их) харуулна. Java-д `ThreadMXBean.getThreadAllocatedBytes` нь **нийт хуваарилсан** санах ойг (хожим чөлөөлөгдсөн ч тоологдоно) харуулна. Энэ хоёр **өөр** хэмжигдэхүүн. Багш асуувал энэ ялгааг хэлээрэй.

2.3 Дахин дуудах (рекурс) ба тэгшитгэл

Функц өөрийгөө жижиг ороллтой дуудахыг **дахин дуудах** гэнэ. Ийм алгоритмын хугацааг **дахин дуудах тэгшитгэлээр** бичнэ:

$$T(n) = a \cdot T(n/b) + f(n)$$

- a — хэдэн дэд бодлого үүсгэх вэ,
- n/b — дэд бодлого бүрийн хэмжээ,
- f(n) — хуваах ба нэгтгэх ажил.

Жишээ: нэгтгэн эрэмбэлэлт хоёр хагас ($a = 2$, $b = 2$) болгож, нэгтгэхэд n алхам хийнэ: $T(n) = 2T(n/2) + \Theta(n)$.

Давталтын мод: тэгшитгэлийг мод болгон зурж, түвшин бүрийн ажлыг нэмнэ (Лаб 2).

Ерөнхий арга (master theorem): f(n)-г $\mathbf{n^{\log_b a}}$ (навчны ажил) -тай харьцуулна (Лаб 3):

Тохиолдол	Нөхцөл	Хариу	Энгийнээр
1	$f(n)$ нь $n^{\log_b a}$-аас полиномоор бага	$\Theta(n^{\log_b a})$	навч (дэд бодлого) давамгайлна
2	$f(n) = \Theta(n^{\log_b a} \cdot \lg^k n)$, $k \geq 0$	$\Theta(n^{\log_b a} \cdot \lg^{k+1} n)$	бүх түвшин ижил, lg n дахин
3	$f(n)$ нь полиномоор их, $a \cdot f(n/b) \leq c \cdot f(n)$, $c < 1$	$\Theta(f(n))$	үндэс (нэгтгэх ажил) давамгайлна

Тэгшитгэл	Хаана	Хариу
$T(n) = 2T(n/2) + \Theta(n)$	нэгтгэн эрэмбэлэлт, LC 53	$\Theta(n \lg n)$
$T(n) = T(n/2) + \Theta(1)$	хоёртын хайлт, LC 50	$\Theta(\lg n)$
$T(n) = 8T(n/2) + \Theta(1)$	дахин дуудах матриц үржвэр	$\Theta(n^3)$
$T(n) = 7T(n/2) + \Theta(n^2)$	Штрассен	$\Theta(n^{\lg 7}) \approx \Theta(n^{2.81})$

2.4 Алгоритм зохиох гурван гол арга

Арга	Санаа	Хэзээ хэрэглэх вэ	Лаб
Хэсэгчлэх (divide and conquer)	хуваах → дэд бодлогуудыг дахин дуудалтаар бодох → нэгтгэх	дэд бодлогууд **давхцахгүй**	2, 3, Бие даалт 1
Динамик программчлал	дэд бодлогын хариуг хүснэгтэд хадгалж, дахин бодохгүй	дэд бодлогууд **давхцдаг**	4
Хомхойлох (greedy)	алхам бүрд «одоо хамгийн сайн» сонголтыг хийгээд буцахгүй	хомхойлох сонголт оновчтой гэж **батлагдсан** үед	5, Бие даалт 2

Холбоо: Фибоначчийг шууд дахин дуудвал нэг утгыг олон дахин боддог (Лаб 1). Энэ нь хэсэгчлэх аргад дэд бодлого давхцах үед гардаг асуудал бөгөөд Лаб 4-ийн динамик программчлал

үүнийг хүснэгтээр шийддэг. Хомхойлох арга (Лаб 5) нь бүр хүснэгт ч хэрэггүй, ганц сонголт хийхэд хангалттай тохиолдол.

2.5 Хэмжилт ба тест

- **Тест** — кодын хариу зөв эсэхийг автоматаар шалгана. Багшийн өгсөн `.tsv` файл (таб-аар тусгаарласан хүснэгт) болон бидний нэмсэн **санамсаргүй тест**: олон санамсаргүй оролтыг удаан боловч найдвартай аргатай (brute force) харьцуулна.
- **Хэмжилт (benchmark)** — хугацааг бодитоор хэмжинэ. Нэг удаа хэмжвэл хэлбэлзэл их тул хэд хэдэн удаа хэмжээд **медиан** (дунд утга) авсан. Маш хурдан функцийг 20 ms болтол давтаж дунджийг авсан.
- **JIT (Java):** Java эхэндээ кодыг удаан тайлбарлаж, олон удаа ажилласан хэсгийг машины код болгон хөрвүүлдэг. Тиймээс хэмжихээс өмнө хэдэн удаа ажиллуулж **халаадаг**.
- **log–log налуу:** log(хугацаа)-г log(n)-ийн эсрэг зурахад шулуун гарах ба налуу нь илтгэгч болно. $\Theta(n^3)$ бол налуу ≈ 3.
- **WSL ба `env_info.sh`:** хэмжилтийг Windows доторх Linux (WSL2 Ubuntu 22.04) дээр хийж, процессор, санах ойн мэдээллийг `env_info.sh`-ээр тайланд хавсаргасан.

2.6 Ажлын урсгал (Git, GitLab)

1. Багшийн санг **fork** хийсэн (Zolnaiz/f.csm301-2627).
2. Лаб бүрд шинэ **салбар** (branch): `git checkout -b lab05`.
3. Өөрийн хавтсанд (`lab05/B190910014/`) код, тест, тайлан бичнэ.
4. `git commit`, `git push` хийж, багшийн сан уруу **нэгтгэх хүсэлт (merge request, MR)** илгээнэ.
5. Багш хамгаалалтын дараа MR-ийг нэгтгэнэ.

Commit-ийн имэйл нь сургуулийн имэйл (`b190910014@must.edu.mn`) байх ёстой.

3 Лаб 1 — Орчин бэлтгэх, Фибоначчи, анаграмм

Таны хавтас: `lab01/B190910014/` — Python код `python/src/`, тест `python/tests/`, Java код `java/src/main/java/com/lab01/`, тайлан `lab01_report.md`. MR !61, нэгтгэгдсэн.

3.1 Энэ лаб юуны тухай вэ

Лаб 1-ийн зорилго нь **ажиллах орчноо бэлдэж**, хичээлийн ажлын урсгалтай танилцах явдал. Үүнд: Linux (WSL) суулгах, код засварлагч тохируулах, багшийн санг татах, тест ажиллуулах. Мөн хоёр жижиг бодлогоор **дахин дуудалт яагаад удаан байж болдгийг** анх удаа мэдэрнэ.

3.2 Орчин (тайлангийн 1–3-р хэсэг)

- `uname -a` — үйлдлийн системийн мэдээлэл (Linux, WSL2 гэдгийг харуулна).
- `lscpu` — процессорын загвар, цөмийн тоо, кэшийн хэмжээ.
- `g++ -version`, `java -version`, `python3 -version` — хөрвүүлэгч, хэлний хувилбар.
- **Vim-ийн тохиргоо:** `noexpandtab` — Tab товч дарахад зай биш жинхэнэ таб тэмдэгт бичнэ. TSV файлд багана таб-аар тусгаарлагддаг тул чухал. `list listchars=tab:>-` — таб-ыг `>--` гэж харагдуулна, ингэснээр зай ба таб-ыг ялгана.

Асуулт: TSV гэж юу вэ? — «Tab-Separated Values». Мөр бүр нэг тохиолдол, багана нь таб-аар тусгаарлагдсан. Тестүүд эдгээр файлаас оролт ба зөв хариуг уншдаг.

3.3 Фибоначчийн тоо

Бодлого. F(0) = 0, F(1) = 1, F(n) = F(n−1) + F(n−2). Дараалал: 0, 1, 1, 2, 3, 5, 8, 13, 21, …

(a) Тодорхойлолтоор шууд бичвэл:

```
# lab01/B190910014/python/src/fibonacci.py
def fibonacci_recursive(n: int) -> int:
    if n < 0:
        raise ValueError(f"n сөрөг байж болохгүй: {n}")
    if n < 2:
        return n                      # F(0) = 0, F(1) = 1
    return fibonacci_recursive(n - 1) + fibonacci_recursive(n - 2)
```

Энэ код зөв боловч **маш удаан**. Яагаад гэдгийг F(5)-ыг бодох дуудалтын модоор харъя:

```
                  F(5)
              /              \
          F(4)                F(3)
         /    \              /    \
     F(3)      F(2)      F(2)     F(1)
    /   \      /  \      /  \
  F(2) F(1)  F(1) F(0) F(1) F(0)
  /  \
F(1) F(0)
```

F(3) хоёр удаа, F(2) гурван удаа бодогдож байна. n томрох тусам давхардал **экспоненциал** өснө. Дуудалтын нийт тоо 2·F(n+1) − 1. $F(n) \approx \varphi^n/\sqrt{5}$ ($\varphi \approx 1.618$ нь алтан харьцаа) тул n нэгээр нэмэгдэх бүрд хугацаа ≈ 1.6 дахин өснө. Python дээр n = 35 үед ≈ 1 секунд, n = 45 үед хэдэн минут болно. Шалгалтын 5 секундэд багтахгүй.

(b) Давталттай хувилбар:

```
# lab01/B190910014/python/src/fibonacci.py
def fibonacci(n: int) -> int:
    if n < 0:
        raise ValueError(f"n сөрөг байж болохгүй: {n}")
    prev, curr = 0, 1                 # F(0), F(1)
    for _ in range(n):
        prev, curr = curr, prev + curr
    return prev
```

Гараар гүйлгэе (n = 5):

Алхам	prev	curr
эхэнд	0	1
1	1	1
2	1	2
3	2	3
4	3	5
5	5	8

Буцаах утга prev = 5 = F(5). Зөвхөн сүүлийн хоёр утгыг санадаг тул **Θ(n) хугацаа, Θ(1) санах ой**.

Логик: рекурс нь «F(n) гэж юу вэ» гэдгийг тодорхойлдог, харин давталт нь «доороос дээш» тооцоолж, утга бүрийг **нэг л удаа** боддог. Энэ бол Лаб 4-ийн динамик программчлалын хамгийн энгийн жишээ.

(c) Буруу оролт. $n < 0$ бол Python-д `ValueError`, Java-д `IllegalArgumentException` шиднэ. Яагаад? Сөрөг дугаартай Фибоначчи тодорхойлогдоогүй тул алдаа гэж мэдэгдэх нь зөв.

(d) n = 46, 47 ба Java-гийн `int`. Java-гийн `int` 32 бит, хамгийн их утга $2^{31} - 1 = 2\,147\,483\,647$. $F(46) = 1\,836\,311\,903$ багтана, харин $F(47) = 2\,971\,215\,073$ багтахгүй. Нэмэх үед **халиж** (overflow) сөрөг тоо ($-1\,323\,752\,223$) гарна. Тиймээс `long` (64 бит) хэрэглэсэн. Python-ы `int` хязгааргүй урттай тул ийм асуудал гарахгүй.

3.4 Анаграмм

Бодлого. Үгсийн жагсаалтаас анаграммуудыг (ижил үсгүүдээс өөр дарааллаар бүтсэн үгс, жишээ нь «eat», «tea», «ate») бүлэглэнэ.

```
# lab01/B190910014/python/src/anagrams.py
def find_anagrams(dictionary: List[str]) -> List[List[str]]:
    sorted_string_to_anagrams = collections.defaultdict(list)
    for s in dictionary:
        sorted_string_to_anagrams["".join(sorted(s))].append(s)   # «eat» → «aet»
    return [
        group
        for group in sorted_string_to_anagrams.values()
        if len(group) >= 2
    ]
```

Санаа: үгийн үсгүүдийг эрэмбэлбэл анаграммууд **ижил** мөр болно («eat» → «aet», «tea» → «aet»). Тэр мөрийг түлхүүр болгож толинд бүлэглэнэ. 2 ба түүнээс олон гишүүнтэй бүлгүүдийг л буцаана.

Асуулт: «Aa» ба «aa» анаграмм уу? — Үгүй. 'A'-ийн код 65, 'a'-ийнх 97 тул эрэмбэлсэн түлхүүр нь «Aa» ба «aa» өөр. Том, жижиг үсгийг ялгадаг.

Хугацаа: m урттай n үгэнд үг бүрийг эрэмбэлэх $\Theta(m \lg m)$, нийт $\Theta(n \cdot m \lg m)$.

Тест. Бид дараах тохиолдлуудыг нэмсэн: том/жижиг үсэг, анаграммгүй жагсаалт, ганц үг, давхардсан үг, урт нь өөр үгс, цифр, кирилл үсэг.

3.5 Багшийн асууж болох асуулт

- **Рекурсив Фибоначчи яагаад удаан бэ?** — Нэг утгыг олон дахин боддог, дуудалтын тоо φ^n шиг экспоненциал өснө.
- **Давталттай нь яагаад хурдан бэ?** — Утга бүрийг нэг удаа, өмнөх хоёроос нь бодно. $\Theta(n)$.
- **F(47)-д юу болдог вэ?** — `int` хэтэрч сөрөг болно. `long` хэрэглэнэ.
- **`sorted(s)` юу буцаадаг вэ?** — Үсгүүдийн эрэмбэлсэн жагсаалт, `.join(...)` түүнийг мөр болгоно.

4 Лаб 2 — Ажиллах хугацааг тодорхойлох

Таны хавтас: `lab02/B190910014/` — Python код `python/src/`, тест `python/tests/`, Java код `java/src/main/java/com/lab02/`, тайлан `lab02_report.md`. MR !60, нэгтгэгдсэн.

4.1 Энэ лаб юуны тухай вэ

Лаб 2 нь **алгоритмын алхмыг тоолж, хугацааг томьёогоор тодорхойлох** аргад сургадаг. Эрэмбэлэх ба хайх алгоритмуудад «энэ мөр хэдэн удаа ажилласан бэ» гэдгийг тоолуураар (shifts, writes, calls) бодитоор тоолж, онолын томьёотой тулгана. Мөн **дахин дуудах тэгшитгэл** ба **давталтын мод**-той анх танилцана. Энэ нь Лаб 3-ын ерөнхий аргын суурь.

4.2 Даалгавар 1 — Зөөх эрэмбэлэлт (insertion sort)

Санаа. Хөзөр гартаа эрэмбэлэхтэй адил: зүүн хэсэг аль хэдийн эрэмбэлэгдсэн. Дараагийн тоог (key) авч, өөрөөсөө их гишүүдийг нэг нэгээр баруун тийш **зөөж**, гарсан зайд key-г тавина.

```
# lab02/B190910014/python/src/insertion_sort.py
def insertion_sort(a):
    shifts = 0
    for i in range(1, len(a)):
        key = a[i]
        j = i - 1
        while j >= 0 and a[j] > key:
            a[j + 1] = a[j]          # зөөлт - энийг тоолно
            shifts += 1
            j -= 1
        a[j + 1] = key
    return a, shifts
```

Гараар гүйлгэе, a = [5, 2, 4, 1]:

i	key	Үйлдэл	Массив	shifts
1	2	5-ыг зөөнө	[2, 5, 4, 1]	1
2	4	5-ыг зөөнө (2 < 4 зогсоно)	[2, 4, 5, 1]	2
3	1	5, 4, 2-ыг зөөнө	[1, 2, 4, 5]	5

Чухал ажиглалт: зөөлтийн тоо = **урвуу хосын тоо**. Урвуу хос гэдэг нь $i < j$ боловч $a[i] > a[j]$ байх хос. [5, 2, 4, 1]-д урвуу хосууд: (5,2), (5,4), (5,1), (2,1), (4,1) — яг 5. Яагаад тэнцүү вэ? Зөөлт бүр яг нэг урвуу хосыг (key ба түүнээс их нэг гишүүн) засдаг.

(b) Урвуу эрэмбэтэй оролтод яагаад $n(n-1)/2$? $[n, n-1, \ldots, 1]$ үед i-р алхамд өмнөх i гишүүн бүгд key-ээс их тул бүгдийг зөөнө: $1 + 2 + \ldots + (n-1) = n(n-1)/2$.

(c) $n(n-1)/2 = \Theta(n^2)$ баталгаа. $c_1 = 1/4$, $c_2 = 1/2$, $n_0 = 2$ гэвэл $n \geq 2$ бүрд $n^2/4 \leq n(n-1)/2 \leq n^2/2$.

- Баруун тал: $n(n-1)/2 = n^2/2 - n/2 \leq n^2/2$ (ойлгомжтой).
- Зүүн тал: $n(n-1)/2 \geq n^2/4 \Leftrightarrow 2n^2 - 2n \geq n^2 \Leftrightarrow n^2 \geq 2n \Leftrightarrow n \geq 2$.

Хамгийн сайн үед (эрэмбэлэгдсэн оролт) зөөлт 0, хугацаа $\Theta(n)$.

4.3 Даалгавар 2 — Нэгтгэн эрэмбэлэлт (merge sort)

Санаа. Массивыг хоёр хувааж, хагас бүрийг дахин дуудалтаар эрэмбэлээд, хоёр эрэмбэлэгдсэн хагасыг **нэгтгэнэ (merge)**. Нэгтгэхдээ хоёр хагасын эхний гишүүдийг харьцуулж, багыг нь авна.

```
# lab02/B190910014/python/src/merge_sort.py
def _merge_sort(a, p, r):
    if p >= r:                          # 0 эсвэл 1 гишүүн - эрэмбэлэгдсэн
```

```
        return 0
    q = (p + r) // 2
    writes = _merge_sort(a, p, q)       # зүүн хагас
    writes += _merge_sort(a, q + 1, r)  # баруун хагас
    return writes + _merge(a, p, q, r)  # нэгтгэх
```

`_merge` нь L = a[p..q], R = a[q+1..r]-г хуулж, `while i < n_L and j < n_R` давталтаар багыг нь a[k] уруу бичнэ. Аль нэг нь дуусмагц нөгөөгийн үлдэгдлийг хуулна.

Нэгтгэх жишээ: L = [2, 5], R = [1, 4]:

Харьцуулалт	Авах	Үр дүн
2 ба 1	1 (R)	[1]
2 ба 4	2 (L)	[1, 2]
5 ба 4	4 (R)	[1, 2, 4]
R дууссан	5 (L-ийн үлдэгдэл)	[1, 2, 4, 5]

(a) Бичилтийн тоо. Merge бүр a[p..r]-ийн гишүүн бүрийг яг нэг удаа бичнэ: $r - p + 1$. Оролтын агуулгаас **хамаарахгүй**.

(b) $n = 2^k$ үед яг $n \cdot k$ яагаад? Давталтын модыг зуръя:

```
түвшин 0:            [ n ]                    нэгтгэх ажил: n
түвшин 1:      [n/2]       [n/2]              n/2 + n/2 = n
түвшин 2:   [n/4] [n/4] [n/4] [n/4]           n
  ...
түвшин k:   [1] [1] ... [1]                   нэгтгэлт байхгүй
```

Түвшин бүрд нийт n гишүүн нэгтгэгдэнэ, нэгтгэлт хийгдэх түвшин $k = \lg n$ ширхэг. Нийт $n \cdot k = \mathbf{n \lg n}$. Тэгшитгэл: $T(n) = 2T(n/2) + \Theta(n) \to \Theta(n \lg n)$.

(c) R-ийн үлдэгдлийг хуулах давталтыг хасвал юу болох вэ?

- **Эрэмбэлэлт эвдрэхгүй.** L эхэлж дуусвал R-д үлдсэн гишүүд a-гийн яг өөрсдийн байранд аль хэдийн зөв дараалалтай байгаа. R-г тэндээс хуулсан бөгөөд тэр хэсгийг хэн ч дарж бичээгүй.
- **Бичилтийн тоо багасна** (өсөх оролтод $24 \to 12$) бөгөөд оролтоос хамаардаг болно.
- **L-ийн давталтыг хасаж болохгүй:** a[p..q]-г үндсэн давталт дарж бичсэн тул L-ийн үлдэгдэл алдагдана.

4.4 Даалгавар 3 — Хоёртын хайлт

Санаа. Эрэмбэлэгдсэн массивын дунд гишүүнийг x-тэй харьцуулна. Тэнцүү бол олдлоо. x их бол баруун хагаст, бага бол зүүн хагаст хайна. Алхам бүрд хайх муж хоёр дахин багасна.

```
# lab02/B190910014/python/src/binary_search.py
def _search(a, x, p, r):
    if p > r:
        return -1, 1                        # хоосон муж - олдсонгүй
    q = (p + r) // 2
    if a[q] == x:
        return q, 1
    if a[q] < x:
        index, calls = _search(a, x, q + 1, r)
    else:
        index, calls = _search(a, x, p, q - 1)
    return index, calls + 1                 # энэ дуудалтыг нэмж тоолно
```

Тэгшитгэл: $T(n) = T(n/2) + \Theta(1) \to \Theta(\lg n)$. Мод нь салаагүй гинж.

Дуудалтын тоо $\lfloor \lg n \rfloor + 2$ (олдохгүй, хамгийн муу үед): хэсгийн урт $n, n/2, n/4, \ldots, 1$ гэж $\lfloor \lg n \rfloor + 1$ удаа, дээр нь $p > r$ болох нэг дуудалт. Жишээ $n = 8$, $x = 99$: урт $8 \to 4 \to 2 \to 1 \to 0$, нийт $5 = \lfloor \lg 8 \rfloor + 2$.

Асуулт: яагаад зүүн тийш явахад цөөн дуудалт гардаг вэ? — Зүүн хэсэг $\lfloor (m-1)/2 \rfloor$, баруун хэсэг $\lfloor m/2 \rfloor$ урттай, зүүн нь богино эсвэл тэнцүү.

4.5 Даалгавар 4 — Хоёртын хайлттай зөөх эрэмбэлэлт

Санаа. Зөөх эрэмбэлэлтэд key-г хаана тавихыг шугаман биш **хоёртын хайлтаар** олно. Тэгээд тэр байрлалаас хойших гишүүдийг зөөнө.

```
# lab02/B190910014/python/src/binary_insertion_sort.py (хэсэг)
lo, hi = 0, i
while lo < hi:
    mid = (lo + hi) // 2
    comparisons += 1
    if a[mid] <= key:        # тэнцүү үед ард нь оруулна - тогтвортой
        lo = mid + 1
    else:
        hi = mid
for j in range(i - 1, lo - 1, -1):
    a[j + 1] = a[j]          # зөөлт
    shifts += 1
a[lo] = key
```

- **`<=` яагаад?** Тэнцүү гишүүдийн **ард** оруулахын тулд. Ингэвэл тэнцүү гишүүдийн анхны дараалал хадгалагдана (тогтвортой эрэмбэлэлт) бөгөөд зөөлтийн тоо энгийн зөөх эрэмбэлэлттэй яг ижил гарна.
- **Харьцуулалт** нийт $\approx \Sigma \lg i \approx n \lg n$ — оролтоос бараг хамаарахгүй.
- **Зөөлт** нь урвуу хосын тоо хэвээр, урвуу оролтод $n(n-1)/2$.
- **Яагаад $\Theta(n^2)$ хэвээр вэ?** Хайлт хурдассан ч гишүүдийг нэг нэгээр нь зөөсөөр байна. Хэмжилтэд n хоёр дахин өсөхөд хугацаа ≈ 4 дахин ($35 \to 147 \to 613 \to 2738$ ms) өссөн.

Логик: хурдасгахын тулд **хамгийн их зардалтай хэсгийг** олох хэрэгтэй. Энд зардал нь хайлт биш, зөөлт.

4.6 Даалгавар 5 — Дарааллын зөрөө (ranking distance)

Бодлого. Хоёр хүн n номыг эрэмбэлжээ (A, B). Хэдэн хос номыг хоёр хүн эсрэгээр эрэмбэлсэн бэ?

Санаа. B-ийн ном бүрийг A дахь байрлалаар нь сольж C дараалал үүсгэнэ. Тэгвэл зөрсөн хос бүр C-д **урвуу хос** болно.

```
A = 1 2 3 4        pos[1]=0, pos[2]=1, pos[3]=2, pos[4]=3
B = 2 1 4 3   →    C = pos[B[i]] = 1 0 3 2
C-ийн урвуу хос: (1,0), (3,2)  →  зөрөө = 2
```

Урвуу хосыг $\Theta(n^2)$-оор биш, **нэгтгэн эрэмбэлэлтийн явцад** тоолно:

```
# lab02/B190910014/python/src/ranking_distance.py (_merge_count-ийн хэсэг)
else:                                  # right[j] < left[i]
```

```
    seq[k] = right[j]
    inversions += n_left - i          # left[i..] бүгд right[j]-ээс их
    j += 1
```

Яагаад n_left − i? L эрэмбэлэгдсэн тул left[i] > right[j] бол left[i], left[i+1], ...-ийн бүгд right[j]-ээс их. Тэд бүгд right[j]-тэй урвуу хос үүсгэнэ.

Тэгшитгэл: $T(n) = 2T(n/2) + \Theta(n) \to \Theta(n \lg n)$. Шууд арга $\Theta(n^2)$ байсан.

Java-д `long` яагаад? $n = 10^5$ үед зөрөө $2.5 \cdot 10^9$ хүрч `int`-ээс хэтэрнэ. Санамсаргүй дараалалд хос бүр $\frac{1}{2}$ магадлалтай урвуу тул дундаж $n(n-1)/4$.

4.7 Онлайн бодлого

- **LC 912 Sort an Array:** зөөх эрэмбэлэлт $\Theta(n^2)$ тул $n = 5 \cdot 10^4$ үед удаан (TLE). Нэгтгэн эрэмбэлэлт $\Theta(n \lg n)$ хангалттай.
- **LC 493 Reverse Pairs:** $i < j$, $nums[i] > 2 \cdot nums[j]$ хосыг тоолно. Нэгтгэхийн өмнө хоёр эрэмбэлэгдсэн хагаст хоёр заагчаар $\Theta(n)$-д тоолоод, дараа нь ердийнхөөр нэгтгэнэ.
- **LC 33 Search in Rotated Sorted Array:** эргүүлсэн массивыг дундуур нь хуваахад нэг тал заавал эрэмбэлэгдсэн. Тэр тал эрэмбэлэгдсэн эсэхийг $nums[p] \le nums[q]$-ээр шалгаад, target түүний хязгаарт байвал тийшээ, эс бөгөөс нөгөө тал руу явна. $\Theta(\lg n)$.

4.8 Багшийн асууж болох асуулт

- **Зөөлтийн тоо юутай тэнцүү вэ, яагаад?** — Урвуу хосын тоо. Зөөлт бүр нэг урвуу хосыг засна.
- **Merge-ийн бичилт яагаад оролтоос хамаарахгүй вэ?** — Гишүүн бүр яг нэг удаа бичигдэнэ.
- **n lg n хаанаас гарсан бэ?** — Түвшин бүрд n ажил, lg n түвшин.
- **Хоёртын хайлттай зөөх эрэмбэлэлт яагаад хурдан биш вэ?** — Зөөлт $\Theta(n^2)$ хэвээр.
- **Зөрөөг яагаад нэгтгэн эрэмбэлэлтээр тоолсон бэ?** — Хоёр хагас эрэмбэлэгдсэн тул нэг харьцуулалтаар олон урвуу хосыг нэг дор тоолж болно (`n_left − i`).

5 Лаб 3 — Хэсэгчлэх арга

Таны хавтас: `lab03/B190910014/` — Python код `python/src/`, тест `python/tests/`, Java код `java/src/main/java/com/lab03/`, тайлан `lab03_report.md`. MR !146.

5.1 Энэ лаб юуны тухай вэ

Лаб 2-т давталтын мод зурж хугацааг гаргаж байсан. Лаб 3 нь үүнийг **ерөнхий арга (master theorem)** гэдэг бэлэн дүрмээр хурдан бодохыг заана. Дараа нь хэсэгчлэх аргыг **матриц үржүүлэх** бодлогод гурван янзаар хэрэглэнэ: энгийн давталт, дахин дуудалт, Штрассен. Мөн Гауссын арга, уншлагын хэсэг, A төвшинд Фибоначчийг матрицаар бодох ба гүйцэтгэлийг сайжруулах ажил бий.

Миний сонгосон төвшин: A (5 бодлого хоёр хэлээр + B бодлогуудыг сайжруулсан).

5.2 Ерөнхий арга — тохиолдлыг яаж тодорхойлох вэ

$T(n) = a \cdot T(n/b) + f(n)$ өгөгдвөл алхам алхмаар:

1. a, b-г олно.
2. **Урсгал функц** $n^{\log_b a}$-г тооцоолно. Энэ нь «модны навчны нийт ажил».

3. f(n)-тэй харьцуулна.

- f нь полиномоор (n^ϵ дахин) **бага** бол 1-р тохиолдол → $\Theta(n^{\log_b a})$.
- f нь яг ижил, эсвэл $\lg^k n$ дахин их ($k \geq 0$) бол 2-р тохиолдол → $\Theta(n^{\log_b a} \cdot \lg^{k+1} n)$.
- f нь полиномоор **их** бөгөөд $a \cdot f(n/b) \leq c \cdot f(n)$ ($c < 1$) бол 3-р тохиолдол → $\Theta(f(n))$.

id	Тэгшитгэл	n^(log_b a)	f(n)	Тохиолдол	Шийд
R1	$2T(n/2) + n$	n	n	2 ($k = 0$)	$\Theta(n \lg n)$
R2	$T(n/2) + 1$	1	1	2 ($k = 0$)	$\Theta(\lg n)$
R3	$8T(n/2) + 1$	n^3	1	1	$\Theta(n^3)$
R4	$3T(n/4) + n^2$	$n^{0.793}$	n^2	3	$\Theta(n^2)$
R5	$4T(n/2) + n^2$	n^2	n^2	2 ($k = 0$)	$\Theta(n^2 \lg n)$
R6	$2T(n/2) + n/\lg n$	n	$n/\lg n$	0 (хамаарахгүй)	$\Theta(n \lg \lg n)$
R7	$7T(n/2) + n^2$	$n^{2.807}$	n^2	1	$\Theta(n^{\lg 7})$
R8	$2T(n/2) + n \lg n$	n	$n \lg n$	2 ($k = 1$)	$\Theta(n \lg^2 n)$

R4 жишээгээр (шинжилгээ (б)): $a = 3$, $b = 4$. $\log_4 3 \approx 0.793$. $f(n) = n^2$ нь $n^{0.793}$-аас $n^{1.2}$ дахин их → 3-р тохиолдол байж болно. **Тогтмол нөхцөл** шалгана: $a \cdot f(n/b) = 3 \cdot (n/4)^2 = (3/16) \cdot n^2 \leq c \cdot n^2$, $c = 3/16 < 1$ ✓. Хариу $\Theta(n^2)$. (Үндэс давамгайлна: дээд түвшний ажил n^2, доод түвшнүүд 3/16 дахин буурна.)

R8: $a = b = 2$, урсгал n. $f(n) = n \lg n = n \cdot \lg^1 n$ → 2-р тохиолдол $k = 1$ → $\Theta(n \lg^2 n)$.

R6 яагаад ерөнхий аргад хамаарахгүй вэ? $f(n) = n/\lg n$ нь n-ээс зөвхөн lg n дахин бага. 1-р тохиолдол **полиномоор** (n^ϵ дахин) бага байхыг шаарддаг, гэтэл lg n нь ямар ч n^ϵ-ээс удаан өсдөг. 2-р тохиолдолд $n/\lg n = n \cdot \lg^{-1} n$, $k = -1 < 0$ тул болохгүй. 3-р нь f их байхыг шаарддаг. Иймд «зааг» дээр унана.

R6-ийн шийдийг давталтын модоор (А төвшин, шинжилгээ (д)): $n = 2^k$ гэе.

- i-р түвшинд 2^i зангилаа, тус бүр $n/2^i$ хэмжээтэй. Нэг зангилааны ажил $(n/2^i)/\lg(n/2^i) = (n/2^i)/(k-i)$.
- Түвшний нийт: $2^i \cdot (n/2^i)/(k-i) = \mathbf{n/(k-i)}$.
- Нийлбэр: $n/k + n/(k-1) + \ldots + n/1 = n \cdot (1 + 1/2 + \ldots + 1/k) = n \cdot H_k$.
- Гармоник тоо $H_k \approx \ln k$, $k = \lg n$ тул $H_k = \Theta(\lg \lg n)$. Хариу $\mathbf{\Theta(n \lg \lg n)}$.

Логик: түвшин бүрийн ажил ижил биш, доошлох тусам удаан (гармоник) өсдөг. Тиймээс 2-р тохиолдлын «бүх түвшин ижил» нөхцөл таарахгүй.

5.3 Даалгавар 1 — Давталттай матриц үржүүлэх

$C = A \cdot B$, $c_{ij} = \Sigma_k\, a_{ik} \cdot b_{kj}$.

```
# lab03/B190910014/python/src/matrix_multiply.py
def matrix_multiply(a, b):
    n = check_square(a, b)      # хэмжээ таарахгүй бол ValueError
    c = zeros(n)                # C-г ТЭГЛЭНЭ
    mults = 0
    for i in range(n):
        for j in range(n):
            for k in range(n):
                c[i][j] = c[i][j] + a[i][k] * b[k][j]
                mults += 1
    return c, mults
```

- **Үржүүлэлт яг n^3:** гурван давталт тус бүр n удаа.

- **C-г яагаад тэглэдэг вэ?** Хийсвэр кодын мөр `c_ij = c_ij + ...` нь C-д **нэмдэг**. Тэглэхгүй бол хоёр дахь дуудалтад өмнөх үр дүн дээр нэмэгдэж буруу гарна. Үүнийг тусгай тестээр шалгасан.
- **Хэмжээ таарахгүй бол** (2×2 ба 3×3) Python `ValueError`, Java `IllegalArgumentException`.

5.4 Даалгавар 2 — Дахин дуудах матриц үржүүлэх

Санаа. n×n матрицыг 4 ширхэг n/2 × n/2 дэд матрицад хуваана:

```
| C11 C12 |   | A11 A12 |   | B11 B12 |
| C21 C22 | = | A21 A22 | · | B21 B22 |

C11 = A11·B11 + A12·B21      C12 = A11·B12 + A12·B22
C21 = A21·B11 + A22·B21      C22 = A21·B12 + A22·B22
```

Нийт **8** дэд үржвэр. Дэд матрицыг **хуулахгүй**, зүүн дээд булангийн товъёг (i, j, k)-оор дамжуулна:

```
# lab03/B190910014/python/src/matrix_multiply_recursive.py
def _recurse(a, b, c, i, j, k, n):
    """C[i.., j..] += A[i.., k..] · B[k.., j..] (n×n дэд матрицууд)."""
    if n == 1:
        c[i][j] = c[i][j] + a[i][k] * b[k][j]
        return 1
    h = n // 2
    mults = 0
    mults += _recurse(a, b, c, i,     j,     k,     h)  # C11 += A11·B11
    mults += _recurse(a, b, c, i,     j + h, k,     h)  # C12 += A11·B12
    mults += _recurse(a, b, c, i + h, j,     k,     h)  # C21 += A21·B11
    mults += _recurse(a, b, c, i + h, j + h, k,     h)  # C22 += A21·B12
    mults += _recurse(a, b, c, i,     j,     k + h, h)  # C11 += A12·B21
    mults += _recurse(a, b, c, i,     j + h, k + h, h)  # C12 += A12·B22
    mults += _recurse(a, b, c, i + h, j,     k + h, h)  # C21 += A22·B21
    mults += _recurse(a, b, c, i + h, j + h, k + h, h)  # C22 += A22·B22
    return mults
```

Эхний 4 дуудалт нь k-г хэвээр (A-ийн зүүн хагас × B-ийн дээд хагас), сүүлийн 4 нь k + h (A-ийн баруун хагас × B-ийн доод хагас). C-ийн дөрвөн хэсэг тус бүрд хоёр удаа нэмэгдэнэ.

Яагаад гурван товъёг хангалттай вэ? C-ийн мөр нь A-ийн мөртэй (i), C-ийн багана нь B-ийн баганатай (j), A-ийн багана нь B-ийн мөртэй (k) давхцдаг.

Тэгшитгэл: $T(n) = 8T(n/2) + \Theta(1) = R3 \to \Theta(n^3)$. Үржүүлэлтийн тоо давталттайтай яг ижил n^3.

(в) Дэд матрицыг хуулбарладаг байсан бол? Хуулах нь $\Theta(n^2)$ тул $T(n) = 8T(n/2) + \Theta(n^2)$. n^3 ба n^2-ыг харьцуулахад n^2 нь n^3-аас n^1 дахин бага → 1-р тохиолдол хэвээр, $\Theta(\mathbf{n^3})$ **өөрчлөгдөхгүй**. Зөвхөн тогтмол үржүүлэгч өснө.

5.5 Даалгавар 4 (B) — Штрассены алгоритм

Санаа. 8 үржвэрийн оронд **7** үржвэрээр ($P_1 \ldots P_7$) болгож, нэмэх/хасах үйлдлээ нэмэгдүүлнэ. Нэмэх нь $\Theta(n^2)$, үржүүлэх нь их зардалтай тул ашигтай.

- Алхам 1: A, B-г 4 дэд матрицад хуваах.
- Алхам 2: 10 нийлбэр/ялгавар $S_1 \ldots S_{10}$ (жишээ нь $S_1 = B_{12} - B_{22}$).
- Алхам 3: 7 үржвэрийг дахин дуудалтаар: $P_1 = A_{11} \cdot S_1$, …, $P_7 = S_9 \cdot S_{10}$.

• Алхам 4: $C_{11} = P_5 + P_4 - P_2 + P_6$, $C_{12} = P_1 + P_2$, $C_{21} = P_3 + P_4$, $C_{22} = P_5 + P_1 - P_3 - P_7$.

(г) Нэмэх/хасах үйлдлийн тоо: S-д 10, C-д $3 + 1 + 1 + 3 = 8$, нийт **18** удаа $n/2 \times n/2$ матриц. Тус бүр $(n/2)^2$ гишүүнтэй тул $18 \cdot n^2/4 = 4.5n^2 = \Theta(n^2)$. Тэгшитгэл $T(n) = 7T(n/2) + \Theta(n^2) = R7 \rightarrow \Theta(\mathbf{n^{lg\ 7}}) \approx \Theta(\mathbf{n^{2.807}})$.

Үржүүлэлтийн тоо яг $\mathbf{7^{lg}\ n = n^{lg}\ 7}$. $n = 1024$ үед энгийн $2^{30} \approx 1.07 \cdot 10^9$, Штрассен $7^{10} \approx 2.8 \cdot 10^8$ — 3.8 дахин цөөн.

Гэвч бодит хугацаанд Штрассен удаан байсан! Java $n = 512$ үед дахин дуудах аргаас 57 дахин удаан. Яагаад? $n = 1$ хүртэл хуваахад дуудалт бүр 18 нэмэх үйлдэл, 26 орчим түр матриц үүсгэдэг. Нэг скаляр үржвэрт ногдох энэ зардал асар их. Онолоор огтлолцох цэг $n_0 \approx 10^{11}$ — ийм матриц санах ойд багтахгүй.

Шийдэл — cutoff (А төвшний сайжруулалт): $n \leq 64$ (эсвэл 32) болмогц энгийн давталттай аргад шилжинэ.

```
# lab03/B190910014/python/src/strassen.py (_strassen-ийн хэсэг)
if n <= cutoff:
    return matrix_multiply(a, b)
```

Үр дүн: Java $n = 512$ үед цэвэр Штрассенаас **99 дахин** хурдан бөгөөд энгийн аргаас ч хурдан. Хуваарилсан санах ой 9.3 GB → 149 MB. Үржүүлэлт нь цөөрөөгүй, бүр өссөн (40 сая → 79 сая) атлаа хурдан. **Сургамж:** хугацааг үржүүлэлтийн тоо биш, нийт ажил (түр матриц, дуудалт, кэш) тодорхойлдог.

5.6 Даалгавар 3 ба 5 — Матриц–вектор, Гауссын арга

matrix_vector: $y_i = \Sigma_j\ a_{ij}x_j$. Мөр бүрд n үржүүлэх + n нэмэх тул $\mathbf{2n^2}$ үйлдэл.

gauss_solve (B): $Ax = b$ системийг бодно.

1. **Шууд явц:** k-р баганын доорх гишүүдийг тэглэнэ. $i > k$ мөр бүрд $\text{factor} = a_{ik}/a_{kk}$ (1 хуваах), тэгээд мөрөөс factor × k-р мөрийг хасна (гишүүн бүрд үржүүлэх + хасах = 2).
2. **Буцах орлуулга:** доороос дээш $x_i = (b_i - \Sigma_{j>i}\ a_{ij}x_j)/a_{ii}$.

Нийт үйлдэл $(4n^3 + 15n^2 - 7n)/6 \approx \frac{2}{3}n^3$. Мөрийн хувиргалт (pivoting) хийхгүй, учир нь матрицууд **диагональ давамгай** (a_{ii} нь мөрийн бусад гишүүдийн үнэмлэхүй утгын нийлбэрээс их) тул хувиргалтын гишүүн тэг болохгүй.

Уншлагын асуултууд:

- **(i) $\mathbf{O(n^3)}$ нь ямар хэмжигдэхүүн вэ?** Ньютоны аргын **нэг алхам** дахь арифметик үйлдлийн тоо.
- **(ii) n нь юу вэ?** Үл мэдэгдэгчийн (тэгшитгэлийн) тоо.
- **(iii) $\mathbf{n^3}$ нь ямар үйлдэл вэ?** n×n систем бодох Гауссын үйлдлүүд ($\approx \frac{2}{3}n^3$).
- **$n = 1000$ үед:** нэг Гаусс $\approx 6.7 \cdot 10^8$ үйлдэл, таван матриц–вектор үржүүлэлт 10^7 → **67 дахин** цөөн.
- **Энэ нь матриц үржүүлэх бодлогыг өөрчилсөн үү?** Үгүй. Нэг алхмын өртгийг өөр алгоритмаар (вектор үйлдэл) бууруулсан хэрэг.
- **Үл тооцсон хүчин зүйл:** давталтын тоо. Нэг алхам хямдарсан ч алхам олон болвол нийт өртөг буурахгүй.

5.7 Уншлага (в) — Фибоначчи матрицын зэргээр (A)

$M = [[1, 1], [1, 0]]$ бол $M^n = [[F(n+1), F(n)], [F(n), F(n-1)]]$. Тиймээс $F(n)$-ийг олохын тулд M^n-ийг олно. Зэрэг дэвшүүлэхдээ хэсэгчилнэ:

$$M^n = (M^{\lfloor n/2 \rfloor})^2,\ n \text{ сондгой бол дахин M-ээр үржүүлнэ.}$$

```
# lab03/B190910014/python/src/fib_matrix.py
MOD = 10**9 + 7
M = [[1, 1], [1, 0]]
IDENTITY = [[1, 0], [0, 1]]

def _mul(x, y):
    c, mults = matrix_multiply(x, y)                # 1-р бодлогын функц
    return [[v % MOD for v in row] for row in c], mults

def fib_matrix(n):
    if n < 0:
        raise ValueError("n сөрөг байж болохгүй")
    def power(k):
        if k == 0:
            return IDENTITY, 0
        half, mults = power(k // 2)                 # дэд бодлогыг ганцхан удаа
        square, m2 = _mul(half, half)
        mults += m2
        if k % 2 == 1:
            square, m3 = _mul(square, M)
            mults += m3
        return square, mults
    mk, mults = power(n)
    return mk[0][1], mults                          # M^n-ийн баруун дээд = F(n)
```

Жишээ: $M^{13} = (M^6)^2 \cdot M$, $M^6 = (M^3)^2$, $M^3 = (M^1)^2 \cdot M$. Ердөө 4 түвшин. $T(n) = T(n/2) + \Theta(1) \to \Theta(\lg n)$ 2×2 үржүүлэлт. $n = 10^{18}$ үед ердөө 84 матриц үржүүлэлт.

$\Theta(\lg n)$ юуг хэмжиж байна вэ? (i) 2×2 матриц үржүүлэлтийн тоо. (ii) n нь Фибоначчийн дугаар (оролтын урт биш). Оролтын урт $\lg n$ бит тул энэ нь оролтоор **шугаман**. (iii) mod 10^9+7 авдаг тул тоо бүр машины нэг үгэнд багтаж, үржүүлэлт бүр $\Theta(1)$. mod авахгүй бол $F(n) \approx 0.69n$ бит урттай болж, үржүүлэлт хямд байхаа болино.

5.8 A төвшин — Гауссын сайжруулалт ба Якоби

- **gauss_solve_fast:** k-р баганын доорх гишүүд заавал 0 болох тул тэдгээрийг тооцоолохгүй, шууд 0 гэж онооно. Үйлдэл $n(n-1)$-ээр цөөрнө. Python-д мөрийг нэг дор шинэчилснээр 1.4–1.9 дахин хурдан. Java-д ялгаа бараг алга, учир нь JIT аль хэдийн оновчилдог.
- **Якобийн давталт (илүү энгийн арга):** $x_i^{(t+1)} = (b_i - \Sigma_{j \neq i}\, a_{ij} x_j^{(t)})/a_{ii}$. Нэг давталт $2n^2$ (нэг матриц–вектор үржүүлэлт). Диагональ давамгай матрицад нийлэх нь баталгаатай. Манай өгөгдөлд 11–16 давталтаар нийлсэн: $n = 512$ үед Гауссаас **15.6 дахин** цөөн үйлдэл, Python-д 62 дахин хурдан.
- **Холбоо:** энэ нь уншлагын «нэг алхмыг n^3-аас n^2 болгох» санааг бодитоор харуулдаг. Гэхдээ давталтын тоо их бол (жишээ нь бүх гишүүн эерэг үед) нийлэхгүй — уншлагын (e) хэсгийн анхааруулга.

5.9 Онлайн бодлого

- **LC 50 Pow(x, n):** $x^n = (x^{\lfloor n/2 \rfloor})^2$ (сондгой бол $\cdot x$). Дэд бодлогыг **ганц** удаа бодно $\to \Theta(\lg n)$. n сөрөг бол $1/x^{-n}$.
- **LC 53 Maximum Subarray:** хэсэгчлэх: хамгийн их нийлбэртэй дэд массив зүүн хагаст, баруун хагаст эсвэл дунд цэгийг огтолж гарна. $T(n) = 2T(n/2) + \Theta(n) \to \Theta(n \lg n)$. Kadane-ийн шугаман арга $\Theta(n)$ — LeetCode-д 22 дахин хурдан.
- **CF 559B Equivalent Strings:** шууд тодорхойлолтоор 4 дэд бодлого: $T(n) = 4T(n/2) + \Theta(n) \to \Theta(n^2)$, удаан. Каноник хэлбэр (хагас бүрийн каноник хэлбэрийг гаргаад бага нь эхэнд) нь 2 дэд бодлого: $T(n) = 2T(n/2) + \Theta(n) \to \Theta(n \lg n)$.

5.10 Багшийн асууж болох асуулт

- **R4-ийг яаж бодсон бэ? Тогтмол нөхцөл юу вэ?** — Дээрх 4.2.
- **R6 яагаад ерөнхий аргад хамаарахгүй вэ?** — f нь урсгалаас зөвхөн lg n дахин бага, полиномоор биш.
- **Дахин дуудах матриц үржвэр яагаад $\Theta(n^3)$ вэ?** — $8T(n/2) + \Theta(1)$, 1-р тохиолдол.
- **Штрассен 7 үржвэр ашигладаг бол яагаад удаан байсан бэ?** — Нэмэх үйлдэл ба түр матрицын тогтмол зардал. cutoff-оор засна.
- **C-г яагаад тэглэдэг вэ?** — Хийсвэр кодын мөр C-д нэмдэг.
- **fib_matrix-д power(k//2)-г хоёр удаа дуудвал юу болох вэ?** — $T(n) = 2T(n/2) + 1 = \Theta(n)$ болж ашиггүй.

6 Лаб 4 — Динамик программчлал

Таны хавтас: `lab04/B190910014/` — Python код `python/src/`, тест `python/tests/`, Java код `java/src/main/java/com/lab04/`, тайлан `lab04_report.md`. MR !154.

6.1 Энэ лаб юуны тухай вэ

Лаб 1-ийн Фибоначчид нэг утгыг олон дахин боддог асуудал гарсан. Лаб 3-ын хэсэгчлэх аргад дэд бодлогууд **давхцахгүй** байсан. Харин дэд бодлогууд **давхцдаг** үед хэсэгчлэх арга экспоненциал болно. **Динамик программчлал (ДП)** нь дэд бодлого бүрийг **нэг удаа** бодож, хариуг хүснэгтэд хадгалдаг.

ДП хэрэглэх хоёр нөхцөл:

1. **Оновчтой дэд бүтэц:** бодлогын оновчтой шийд нь дэд бодлогуудын оновчтой шийдээс бүрдэнэ.
2. **Давхцах дэд бодлогууд:** ижил дэд бодлого олон дахин гарна.

ДП бичих 4 алхам: (1) дэд бодлогыг тодорхойлох, (2) хариуны хоорондын холбоо (томьёо), (3) хүснэгтийг зөв дарааллаар (жижгээс том руу) бөглөх, (4) шийдийг сэргээх.

6.2 Даалгавар 1 — Ган саваа (rod cutting)

Бодлого. n урттай савааг хэсэглэж зарна. i урттай хэсгийн үнэ p_i. Хамгийн их орлогыг ол.

Дэд бодлого: r[j] — j урттай саваны хамгийн их орлого. **Томьёо:** эхний хэсгийн урт i-г сонговол үлдсэн $j - i$ хэсгийг оновчтой зарна:

$$r[j] = \max(p_i + r[j - i]),\ i = 1, \ldots, j;\ r[0] = 0.$$

```
# lab04/B190910014/python/src/cut_rod.py
def bottom_up_cut_rod(p):
    n = len(p)
    r = [0] * (n + 1)  # r[j]: j урттай саваны хамгийн их орлого
    loops = 0
    for j in range(1, n + 1):
        q = float("-inf")
        for i in range(1, j + 1):  # эхний хэсгийн урт i
            q = max(q, p[i - 1] + r[j - i])
            loops += 1
        r[j] = q
    return r[n], loops             # loops нь (r) шинжилгээний n(n+1)/2
```

Жишээ: p = [1, 5, 8, 9] (урт 1, 2, 3, 4-ийн үнэ).

j	Боломжууд (p_i + r[j−i])	r[j]	Хамгийн сайн
1	1 + 0	1	1
2	1 + 1, 5 + 0	5	2
3	1 + 5, 5 + 1, 8 + 0	8	3
4	1 + 8, 5 + 5, 8 + 1, 9 + 0	**10**	2 + 2

Хугацаа: дотоод давталт j удаа, нийт $1 + 2 + \ldots + n = \mathbf{n(n+1)/2} = \mathbf{\Theta(n^2)}$ (шинжилгээ (г)). Санах ой $\Theta(n)$.

Даалгавар 2 — шийдийг сэргээх (extended): s[j]-д эхний хэсгийн оновчтой уртыг хадгална. Дараа нь n-ээс эхлэн s[n]-ийг таслаад, n − s[n]-ээр үргэлжлүүлнэ. Жишээнд s[4] = 2, s[2] = 2 → хэсгүүд [2, 2].

6.3 Шинжилгээ

(а) SRTBOT хэлбэр:

- **S**ubproblems (дэд бодлого): r[j], j = 0, …, n.
- **R**elate (холбоо): $r[j] = \max(p_i + r[j - i])$.
- **T**opological order (дараалал): j = 0, 1, …, n (r[j] нь зөвхөн бага j-ээс хамаарна).
- **B**ase (суурь): r[0] = 0.
- **O**riginal (эх бодлого): r[n].
- **T**ime (хугацаа): n дэд бодлого × тус бүрд $O(n) = \Theta(n^2)$.

(б) Оновчтой дэд бүтэц — «орлуулах» (cut-and-paste) баталгаа: оновчтой шийдэд эхний хэсэг i, үлдсэн j − i хэсгийг ямар нэг аргаар зарсан гэе. Хэрэв үлдсэн хэсгийн зарсан арга оновчтой биш бол түүнийг оновчтойгоор **сольж** нийт орлогыг өсгөж болох байсан — энэ нь анхны шийд оновчтой гэдэгтэй зөрчилдөнө. Иймд үлдсэн хэсэг заавал оновчтой зарагдсан.

(в) Шууд дахин дуудах Cut-Rod-ийн дуудалт $\mathbf{T(n) = 2^n}$ (индукц): $T(0) = 1$, $T(n) = 1 + \Sigma_{j=0}^{n-1} T(j)$. Индукцийн таамаг $T(j) = 2^j$ ($j < n$) гэвэл $T(n) = 1 + (1 + 2 + \ldots + 2^{n-1}) = 1 + (2^n - 1) = 2^n$. Экспоненциал — учир нь ижил r[j]-г олон дахин боддог. ДП нь үүнийг n(n+1)/2 болгосон.

6.4 Даалгавар 3 — LCS (хамгийн урт ерөнхий дэд дараалал)

Бодлого. Хоёр мөрийн хоёуланд нь (дарааллаа хадгалан, заавал зэрэгцээгүй) орох хамгийн урт дэд дарааллын урт. Жишээ: «ABCBDAB» ба «BDCABA» → «BCBA», урт 4.

Дэд бодлого: c[i][j] — x-ийн эхний i, y-ийн эхний j тэмдэгтийн LCS-ийн урт.

Томьёо:

- x[i] = y[j] бол c[i][j] = c[i−1][j−1] + 1 (ижил тэмдэгтийг LCS-д нэмнэ).
- Ялгаатай бол c[i][j] = max(c[i−1][j], c[i][j−1]) (аль нэгийг нь орхино).

Жишээ x = «AB», y = «BA»:

	∅	B	A
∅	0	0	0
A	0	0	1
B	0	1	1

Хугацаа $\Theta(mn)$. Хүснэгтийг мөр мөрөөр бөглөдөг ба зөвхөн өмнөх мөр хэрэгтэй тул **хоёр мөр** хадгалж санах ойг $\Theta(n)$ болгосон.

6.5 Даалгавар 4 (B) — Матрицуудын цуваа үржвэр

Бодлого. $A_1 A_2 \ldots A_n$ үржвэрт хаалтыг яаж тавивал скаляр үржүүлэлт хамгийн цөөн байх вэ? A_i нь $p_{i-1} \times p_i$ хэмжээтэй. p×q ба q×r матрицыг үржүүлэхэд p·q·r үржүүлэлт.

Жишээ: A_1 (10×100), A_2 (100×5), A_3 (5×50).

- $(A_1 A_2)A_3$: 10·100·5 + 10·5·50 = 5 000 + 2 500 = **7 500**.
- $A_1(A_2 A_3)$: 100·5·50 + 10·100·50 = 25 000 + 50 000 = 75 000.

10 дахин ялгаатай! Иймд хаалт чухал.

Дэд бодлого: m[i][j] — $A_i \ldots A_j$-г үржүүлэх хамгийн бага зардал. **Томьёо:** хамгийн сүүлийн үржүүлэлт k дээр хуваагдана гэвэл

$$m[i][j] = \min(m[i][k] + m[k+1][j] + p_{i-1} \cdot p_k \cdot p_j),\ k = i, \ldots, j-1.$$

Дараалал: гинжийн уртаар (2, 3, …, n) бөглөнө, учир нь урт цуваа богино цуваануудаас хамаардаг. s[i][j] = оновчтой k-г хадгалж хаалтыг сэргээнэ. **$\Theta(n^3)$ хугацаа, $\Theta(n^2)$ санах ой.**

6.6 Даалгавар 5 (B) — Хоёр файлын ялгаа (diff)

Санаа. Хоёр файлын мөрүүдийн LCS нь «өөрчлөгдөөгүй» мөрүүд. LCS-д ороогүй A-ийн мөрүүд «−» (устгасан), B-ийнх «+» (нэмсэн). Ингэвэл өөрчлөлтийн тоо хамгийн бага.

c[i][j]-г **сүүлээр** (a[i:], b[j:]) тодорхойлсон. Ингэснээр ялгааг эхнээс нь дараалаар уншиж болно:

```
# lab04/B190910014/python/src/lcs_diff.py (хэсэг)
while i < m and j < n:
    if a[i] == b[j]:
        out.append(" " + a[i]); i += 1; j += 1      # ижил мөр
    elif c[i + 1][j] >= c[i][j + 1]:
        out.append("-" + a[i]); i += 1              # a[i]-г хасахад LCS буурахгүй
    else:
        out.append("+" + b[j]); j += 1
```

$\Theta(mn)$ хугацаа ба санах ой.

6.7 A төвшин — сайжруулалт

Diff → Myers-ийн алгоритм. Засварын графаар (0, 0)-оос (N, M) хүрэх замыг хайна. Баруун алхам = A-аас хасах, доош = B-д нэмэх, диагональ = ижил мөр (үнэгүй). D = хамгийн цөөн хасах + нэмэх. v[k] нь d алхмаар k = x − y диагональ дээр хүрэх хамгийн хол x. d-г 0, 1, 2, … гэж өсгөж (N, M)-д хүрмэгц зогсоно. Хугацаа **$O((N + M) \cdot D)$** — файлууд бага зэрэг ялгаатай (D жижиг) үед маш хурдан. **git diff яг энэ алгоритмыг ашигладаг.** Хэмжилтэд N = 2000 үед Python 37 дахин, Java 28 дахин хурдан, санах ой 4.5–20 дахин бага.

Матрицын цуваа → энгийн шийдэлтэй харьцуулах. $\Theta(n^3)$ ДП-г оновчтой гэж үзээд, PDF-ийн зааврын дагуу **хүснэгтгүй шууд дахин дуудах** хувилбарыг бичиж харьцуулсан. Дуудалтын тоо $C(n) = 1 + 2 \cdot \Sigma\ C(k) \rightarrow \mathbf{C(n) = 3^{n-1}}$. n = 14 үед ДП-ээс 2 200 дахин удаан. Энгийн хувилбарын санах ой $\Theta(n)$ (стек), ДП-гийнх $\Theta(n^2)$. **Хугацаа ба санах ойн солилцоо:** $\Theta(n^2)$ санах ой «зарж» экспоненциал хугацааг $\Theta(n^3)$ болгож байна.

6.8 Онлайн бодлого

- **CF 189A Cut Ribbon:** a, b, c урттай хэсгүүдээр n-ийг хамгийн олон хэсэгт хуваах. Ган савааны хувилбар: dp[j] = 1 + max(dp[j−a], dp[j−b], dp[j−c]), хүрэх боломжгүйг $-\infty$. $\Theta(n)$.

- **LC 72 Edit Distance:** dp[i][j] = word1[:i]-г word2[:j] болгох хамгийн цөөн үйлдэл. Тэнцүү бол dp[i−1][j−1], эс бөгөөс 1 + min(солих, устгах, оруулах). $\Theta(mn)$.
- **LC 300 Longest Increasing Subsequence:** $\Theta(n^2)$ ДП: dp[i] = 1 + max dp[j] (j < i, nums[j] < nums[i]). Сайжруулсан $\Theta(n \lg n)$: tails[k] = k+1 урттай өсөх дэд дарааллын хамгийн бага сүүлийн гишүүн. Шинэ тоо бүрд хоёртын хайлтаар байрыг олно. LeetCode дээр 1052 ms → 10 ms.

6.9 Багшийн асууж болох асуулт

- **ДП ба хэсэгчлэх аргын ялгаа?** — Дэд бодлого давхцдаг эсэх. ДП нь хариуг хадгална.
- **Ган савааны дотоод давталт яагаад n(n+1)/2 удаа вэ?** — j бүрд j удаа, 1 + … + n.
- **Шууд дахин дуудвал яагаад 2^n вэ?** — Индукц (5.3 (в)).
- **LCS-д хоёр мөр хадгалах нь яагаад хангалттай вэ?** — c[i][·] нь зөвхөн c[i−1][·] ба өөрийн мөрөөс хамаарна.
- **Матрицын цуваг яагаад гинжийн уртаар бөглөдөг вэ?** — m[i][j] нь богино цуваануудаас хамаарна.
- **Myers яагаад хурдан бэ?** — Хүснэгтийг бүхэлд нь биш, ялгаа D-тэй пропорциональ хэсгийг л боддог.

7 Лаб 5 — Хомхойлох алгоритм

Таны хавтас: `lab05/B190910014/` — Python код `python/src/`, тест `python/tests/`, Java код `java/src/main/java/com/lab05/`, тайлан `lab05_report.md`. MR !186.

7.1 Энэ лаб юуны тухай вэ

ДП бүх боломжийг хүснэгтээр шалгадаг. **Хомхойлох арга** нь алхам бүрд «одоо хамгийн сайн харагдах» сонголтыг хийгээд **буцааж хардаггүй**. Ихэнхдээ маш хурдан (эрэмбэлэх $\Theta(n \lg n)$ + нэг гүйлт), гэхдээ үргэлж зөв биш. Тиймээс хомхойлох сонголт зөв гэдгийг **батлах** хэрэгтэй. Гол аргачлал нь **солих арга (exchange argument)**: аль нэг оновчтой шийдийг авч, түүний нэг сонголтыг хомхойлох сонголтоор солиход шийд муудахгүй гэдгийг харуулна.

7.2 Даалгавар 1 — Ажиллагааг түүвэрлэх (activity selection)

Бодлого. n ажиллагаа (лекц, уулзалт), i-р нь $[s_i, f_i)$ хугацаанд. Давхцуулахгүйгээр хамгийн олныг сонго.

Хомхойлох дүрэм: хамгийн эрт дуусдаг-ийг сонгоод, түүнтэй давхцахыг хасна, давтана.

```
# lab05/B190910014/python/src/activity.py
def activity_select(s, f):
    order = sorted(range(len(s)), key=lambda i: f[i])   # дуусах хугацаагаар эрэмбэлнэ
    chosen, last_finish = [], None
    for i in order:
        if last_finish is None or s[i] >= last_finish:   # өмнөхтэй давхцахгүй
            chosen.append(i + 1)                          # 1-ээс эхэлсэн дугаар
            last_finish = f[i]
    return chosen
```

Яагаад хамгийн эрт дуусахыг вэ? Эрт дуусах тусам үлдсэн хугацаа их үлдэж, бусад ажиллагаанд илүү зай гарна.

(а) Солих аргаар баталгаа: a_m нь хамгийн эрт дуусдаг ажиллагаа, A нь аль нэг оновчтой шийд, a_j нь A-д хамгийн эрт дуусдаг ажиллагаа гэе. $a_j \neq a_m$ бол A-аас a_j-г хасаж a_m-ийг нэмье. $f_m \leq f_j$

тул a_m нь A-ийн бусадтай давхцахгүй (тэд a_j дуусахаас хойш эхэлдэг). Тоо нь өөрчлөгдөхгүй тул шинэ шийд мөн оновчтой бөгөөд a_m-ийг агуулна. Иймд хомхойлох сонголт хэзээ ч алдагдалгүй.

(б) «Хамгийн богино ажиллагаа» дүрэм буруу — эсрэг жишээ: $[1, 4), [3, 5), [4, 7)$. Хамгийн богино $[3, 5)$ нь нөгөө хоёртой давхцана. Дүрэм түүнийг сонгоод хоёуланг хасна $\rightarrow$ 1 ажиллагаа. Оновчтой нь $\{[1, 4), [4, 7)\} \rightarrow 2$.

(в) Гэвч энэ дүрэм оновчтойн дор хаяж хагасыг сонгоно (A): сонгосон a бүр оновчтой шийдийн **хамгийн ихдээ 2** ажиллагаатай давхцана. Учир нь a-г сонгох үед үлдсэн бүхнээс a хамгийн богино. Тиймээс a-тай давхцах b нь a-гаас богино биш тул a-гийн дотор багтахгүй, a-гийн эхлэл эсвэл төгсгөлийн цэгийг агуулна. Оновчтой шийдийн ажиллагаанууд хоорондоо давхцахгүй тул нэг цэгийг ганцхан нь агуулна. Иймд $|O| \leq 2 \cdot |G|$.

Хугацаа: эрэмбэлэх $\Theta(n \lg n)$ + гүйлт $\Theta(n)$. Санах ой $\Theta(n)$.

7.3 Даалгавар 2 — Хэсэгчилж авч болох үүргэвч (fractional knapsack)

Бодлого. W багтаамжтай үүргэвчинд w_i жинтэй, v_i үнэтэй эд зүйлсийг хийнэ. Эд зүйлийг **хэсэгчилж** (жишээ нь гурилын тал) авч болно. Нийт үнийг хамгийн их болго.

Хомхойлох дүрэм: нэгж жингийн үнэ v_i/w_i-ээр буурахаар эрэмбэлж, багтаамж дуустал бүтнээр нь авна, сүүлийнхийг хэсэгчилнэ.

Жишээ (K1): w = [10, 20, 30], v = [60, 100, 120], W = 50. Харьцаа 6, 5, 4.

Эд зүйл	Харьцаа	Авсан хэмжээ	Үлдсэн багтаамж	Нийт үнэ
1	6	10 (бүтэн)	40	60
2	5	20 (бүтэн)	20	160
3	4	20/30 хэсэг	0	160 + 80 = **240**

Анхаар: эд зүйлийг хэсэгчилж болохгүй (0/1 үүргэвч) бол хомхойлох арга буруу (энэ жишээнд 220 бол зөв). 0/1 хувилбарт ДП хэрэгтэй. Хэсэгчилж болох үед хомхойлох зөв, учир нь харьцаа өндөр эд зүйлийн хэсгийг харьцаа бага эд зүйлийн хэсгээр солих нь хэзээ ч ашигтай биш.

Java-д харьцааг бутархайгүйгээр $v_i \cdot w_j$ ба $v_j \cdot w_i$ гэж (long) харьцуулсан. Хугацаа $\Theta(n \lg n)$.

7.4 Даалгавар 3 — Хариулт мөнгө ба min_coins

make_change (кассчны арга): хамгийн том дэвсгэртээс эхлэн аль болох олныг авна.

```
# lab05/B190910014/python/src/change.py (make_change-ийн хэсэг)
counts = [0] * len(d)
for j in range(len(d) - 1, -1, -1):      # томоос жижиг уруу
    counts[j], x = divmod(x, d[j])        # хэдэн ширхэг, үлдэгдэл
```

Жишээ 3760₹: 1000×3 (үлдэх 760), 500×1 (260), 100×2 (60), 50×1 (10), $10 \times 1 \rightarrow 8$ дэвсгэрт. Хугацаа $\Theta(m)$, m = дэвсгэртийн төрлийн тоо.

min_coins (ДП): best[y] = 1 + min best[y − d_j]. $\Theta(m \cdot x)$. Энэ нь ямар ч системд зөв.

7.5 Уншлага — каноник систем

Каноник систем гэдэг нь хомхойлох арга **үргэлж** хамгийн цөөн дэвсгэрт өгдөг систем. Каноник биш жишээ: {1, 3, 4}-д 6 = 4 + 1 + 1 (3) гэж хомхойлно, гэтэл 3 + 3 (2).

Козен–Заксын теорем: $c_3 < x < c_(m-1) + c_m$ мужид эсрэг жишээ байхгүй бол систем каноник. Иймд хязгааргүй олон x биш, хязгаартай мужийг шалгахад хангалттай.

- **(а) Төгрөг** {1, 5, 10, 20, 50, 100, 500, 1000, 5000, 10000, 20000}: x = 11 … 29 999 (29 989 дүн) бүгдэд хомхойлох = ДП → **каноник**.
- **{1, 5, 10, 20, 25, 50}:** x = 40 үед хомхойлох 25 + 10 + 5 (3), ДП 20 + 20 (2) → **каноник биш**, хамгийн бага эсрэг жишээ 40.
- **(б) «Дүн бүр өмнөхөөсөө ≥ 2 дахин их бол оновчтой» худал:** {1, 4, 9}, x = 12: хомхойлох 9 + 1 + 1 + 1 (4), оновчтой 4 + 4 + 4 (3).
- **(в) {1, 5, 10, 20, 50} каноник — солих аргаар:** оновчтой шийдэд $1 \le 4$ ширхэг (5 ширхэг 1 → нэг 5), $5 \le 1$ (2×5 → 10), $10 \le 1$ (2×10 → 20), $20 \le 2$ (3×20 → 50 + 10), 20×2 ба 10 хамт байхгүй (→ 50). Тэгвэл 50-аас бага дэвсгэртийн нийлбэр $\le 4 + 5 + 40 = 49 < 50$. Тиймээс 50-ын тоо заавал $\lfloor x/50 \rfloor$ буюу хомхойлох аргатай ижил. Дараагийн дэвсгэрт бүрд адилхан.

7.6 Даалгавар 4 (B) — Хаффманы код

Бодлого. Тэмдэгт бүрд 0/1 код өгч, текстийг аль болох богино битээр кодлох. Код **угтваргүй** (нэг код нөгөөгийн эхлэл биш) байх ёстой, тэгвэл тайлалт нэг утгатай.

Санаа: олон давтагдах тэмдэгтэд богино код. **Хомхойлох алхам:** хамгийн бага давтамжтай **хоёр** зангилааг нийлүүлж нэг зангилаа болгоно (давтамж нь нийлбэр). Дахин давтана. Бага утгын дараалал (heap) ашиглана.

CLRS-ийн жишээ (H1): a:45, b:13, c:12, d:16, e:9, f:5.

Алхам	Хамгийн бага хоёр	Шинэ зангилаа
1	f:5, e:9	14
2	c:12, b:13	25
3	14, d:16	30
4	25, 30	55
5	a:45, 55	100 (үндэс)

Код: a = 0 (1 бит), b, c, d = 3 бит, e, f = 4 бит. Нийт 45·1 + (13 + 12 + 16)·3 + (9 + 5)·4 = **224 бит** (тогтмол 3 битээр 300).

```
# lab05/B190910014/python/src/huffman.py (huffman_build-ийн хэсэг)
heap = [(f, next(tie), sym) for sym, f in freq.items()]
heapq.heapify(heap)
while len(heap) > 1:
    f1, _, left = heapq.heappop(heap)
    f2, _, right = heapq.heappop(heap)
    heapq.heappush(heap, (f1 + f2, next(tie), (left, right)))
```

`tie` — ижил давтамжтай үед heap зангилааг өөр хооронд нь харьцуулахгүй байх дарааллын дугаар. Модыг тойрч зүүн тийш «0», баруун тийш «1» гэж код үүсгэнэ. Ганц тэмдэгттэй үед код «0».

Монгол текст (mn_text.txt): 60 122 тэмдэгт, 50 төрөл. UTF-8 890 336 бит, Хаффман **279 836 бит** (31%), энтропи (онолын доод хязгаар) 277 786 — ердөө 0.7% зөрүү.

Тайлах (анхны): битийг нэг нэгээр нэмж, угтвар нь код болмогц тэмдэгт гаргана.

7.7 Даалгавар 5 (B) — Оновчтой кэш (furthest-in-future)

Бодлого. k хуудас багтаах кэш. Хуудас кэшид байхгүй бол **алдаа** (miss). Кэш дүүрсэн бол аль хуудсыг хасах вэ? Алдааг хамгийн цөөн болго.

Хомхойлох дүрэм: дараагийн хандалт нь **хамгийн хойно** байгаа (эсвэл огт хандагдахгүй) хуудсыг хасна. Энэ нь ирээдүйг мэддэг (offline) оновчтой дүрэм.

Жишээ (P1): k = 2, хандалт 1 2 3 1 2 3 1 2 3.

Хандалт	Кэш (өмнө)	Алдаа?	Хасах	Шалтгаан
1	{}	тийм	—	хоосон зай
2	{1}	тийм	—	хоосон зай
3	{1, 2}	тийм	2	1 нь 4-р байрлалд, 2 нь 5-р — 2 хойно
1	{1, 3}	үгүй	—	
2	{1, 3}	тийм	1	3 нь 6-р, 1 нь 7-р — 1 хойно
3	{2, 3}	үгүй	—	
1	{2, 3}	тийм	3	2 нь 8-р, 3 нь 9-р
2	{1, 2}	үгүй	—	
3	{1, 2}	тийм	аль ч	цаашид хандалтгүй

Нийт **6** алдаа (FIFO, LRU нь 9). Хувийн өгөгдөлд (200 000 хандалт, k = 64) furthest-in-future 59 853, LRU 123 146 — 2 дахин цөөн.

Анхны хэрэгжүүлэлт: хасах бүрд кэш дэх k хуудас бүрийн дараагийн хандалтыг цаашаа шугаман хайлтаар олно → хамгийн муу үед $O(n^2 \cdot k)$.

7.8 A төвшин — сайжруулалт

Кэш:

1. **next_use[i]** — requests[i] хуудасны дараагийн хандалтын байрлалыг төгсгөлөөс эхэн уруу нэг гүйж $\Theta(n)$-д олно. «Дараагийн хандалт хэзээ?» гэсэн асуулт O(n) хайлтаас O(1) болно.
2. Кэш дэх хуудсуудыг дараагийн хандалтаар нь **хамгийн их овоо**нд (max-heap) хадгална. Хуудсанд дахин хандахад шинэ утга нэмж, хуучныг «хуучирсан» гэж үлдээнэ (lazy deletion). Хасахдаа хуучирсныг алгасаж оройг авна.
3. Овоо 4k + 64-өөс томровол шинээр байгуулж O(k) хэмжээнд барина.

Нийт **O(n lg k)**. n = 200 000 үед Python 3.2 c → 0.19 c, Java 383 → 22 ms (**17 дахин**). Төлөөс нь санах ой: O(k)-оос O(n) (next_use массив).

Хаффманы тайлалт:

- **Python:** модыг төлөвт автомат болгож, (төлөв, 8 бит) → (гарах тэмдэгтүүд, шинэ төлөв) хүснэгт байгуулна. 8 бит тутамд ганц хайлт (≈ 1.7 тэмдэгт). 2.6–3.3 дахин хурдан.
- **Java:** модыг int массивт хадгалж, бит бүрд ганц массив хандалтаар алхана (бит бүрд шинэ String үүсгэхгүй). 3.7–5.8 дахин хурдан, санах ойн хуваарилалт 15 дахин бага.

7.9 Онлайн бодлого

- **CF 158B Taxi:** 4 хүнтэй бүлэг бүрд такси; 3 + 1; 2 + 2; үлдсэн 2 + 1 + 1; үлдсэн 1-ийг 4-өөр. $\Theta(n)$.
- **LC 435 Non-overlapping Intervals:** хасах хамгийн цөөн = n − (давхцалгүй хамгийн олон) = ажиллагааг түүвэрлэх бодлого. Accepted, 51 ms.
- **LC 621 Task Scheduler:** хамгийн олон давтагдсан ажлын тоо f, ийм ажлын төрөл c: хариу $\max(N, (f-1)(n+1) + c)$. Accepted, 16 ms.

7.10 Багшийн асууж болох асуулт

- **Хомхойлох арга ДП-ээс юугаараа ялгаатай вэ?** — Нэг сонголт хийгээд буцахгүй, бүх боломжийг шалгахгүй.
- **Хамгийн эрт дуусах дүрэм яагаад зөв бэ?** — Солих аргаар (6.2).

- **Хэсэгчилсэн үүргэвчинд хомхойлох зөв, 0/1-д яагаад буруу вэ?** — 0/1-д эд зүйлийг хувааж чадахгүй тул харьцаа өндрийг авсан ч багтаамж дүүрэхгүй үлдэж болно.
- **Төгрөгийн систем каноник гэдгийг яаж шалгасан бэ?** — Козен–Заксын мужийн 29 989 дүнг ДП-тэй харьцуулсан.
- **Хаффманы код яагаад угтваргүй вэ?** — Тэмдэгт бүр модны навч. Нэг навчны зам нөгөө навчны замын эхлэл байж чадахгүй.
- **Furthest-in-future бодит системд яагаад хэрэглэгддэггүй вэ?** — Ирээдүйг мэдэх боломжгүй. LRU хэрэглэдэг.

8 Бие даалт — хамгаалалтын бэлтгэл

8.1 Хамгаалалт хэрхэн явагдах вэ

Home_Work-ийн дүрмээр:

- Багш таны сонгосон төвшний (таных **A** — бүх 5 бодлого) бодлогоос **санамсаргүй нэгийг** сонгоно.
- Эхлээд бодлогыг **товч танилцуулж**, ямар аргаар бодохоо хэлнэ.
- Код бичих явцдаа мөр бүрээр **юу хийх гэж байгаагаа урьдчилж хэлээд** бичнэ.
- **15 минутад** бодож шалгалтыг давуулна. Давуулж чадахгүй бол дахин 2 оролдлого (нийт 3).
- Тайлангийн үг үсгийн алдаа бүрд −1 оноо. Улаанаар зурагдсан гадаад үгийн утгыг тайлбарлаж чадвал оноо хасахгүй (жишээ нь «inorder», «Python-д», «Моррис»).

Тайлбарлах загвар (бодлого бүрд):

1. «Энэ бодлого ... -ийн олохыг шаардаж байна.»
2. «... аргаар бодно, учир нь ...»
3. Кодыг бичихдээ: «эхлээд ... хадгална», «давталтаар ... », «... бол буцаана».
4. «Хугацаа нь ..., учир нь ...; санах ой ...»

Доорх код бүр **таны өөрийн хавтсан дахь код** (урт тайлбар мөр болон зарим газар хэмжилтийн тоолуурыг хассан, логик нь яг ижил). Бие даалт 1: `prj1/B190910014/` (MR !153), Бие даалт 2: `prj2/B190910014/` (MR !200). Файлын замыг код бүрийн эхэнд бичсэн. Танхимд энэ кодыг өөрөө бичих тул санааг нь ойлгоод, хоосон редакторт хэд хэдэн удаа бичиж дасгал хийгээрэй.

8.2 Бие даалт 1 — Хэсэгчлэх ба дахин дуудах арга (`prj1/B190910014`)

1. Majority Element (LC 169) — хэсэгчлэх. Массивт $\lfloor n/2 \rfloor$-аас олон удаа орсон гишүүнийг ол.

«Олонх гишүүн нь зүүн эсвэл баруун хагасын ядаж нэгд нь олонх байна. Тиймээс хоёр хагасын олонхыг дахин дуудалтаар олоод, ижил бол тэр, өөр бол хоёуланг нь бүхэл мужид тоолж олон удаа орсныг нь сонгоно.»

```
# prj1/B190910014/python/src/p1_majority_element.py
class Solution:
    def majorityElement(self, nums: List[int]) -> int:
        def majority(lo: int, hi: int) -> int:  # nums[lo:hi]-ийн олонх гишүүн
            if hi - lo == 1:
                return nums[lo]
            mid = (lo + hi) // 2
            left = majority(lo, mid)
            right = majority(mid, hi)
            if left == right:
                return left
            # Олонх гишүүн аль нэг хагаст заавал олонх байна - хоёр нэр дэвшигчийг тоолно.
            left_count = sum(1 for i in range(lo, hi) if nums[i] == left)
```

```
            right_count = sum(1 for i in range(lo, hi) if nums[i] == right)
            return left if left_count > right_count else right

        return majority(0, len(nums))
```

$T(n) = 2T(n/2) + \Theta(n) \to \mathbf{\Theta(n \lg n)}$, санах ой $\Theta(\lg n)$ (стек). Массивыг зүсэхгүй, хил (lo, hi)-г л дамжуулдаг.

2. Sorted Array → BST (LC 108) — дахин дуудах.

«Дунд гишүүнийг үндэс болгоно. Зүүн тал нь бүгд бага, баруун тал нь бүгд их тул хайлтын модны нөхцөл биелнэ. Хоёр талын хэмжээ хамгийн ихдээ 1-ээр ялгаатай тул тэнцвэртэй.»

```
# prj1/B190910014/python/src/p2_sorted_array_to_bst.py
class Solution:
    def sortedArrayToBST(self, nums: List[int]) -> Optional[TreeNode]:
        def build(lo: int, hi: int) -> Optional[TreeNode]:  # nums[lo..hi]
            if lo > hi:
                return None
            mid = (lo + hi) // 2
            return TreeNode(nums[mid], build(lo, mid - 1), build(mid + 1, hi))

        return build(0, len(nums) - 1)
```

$T(n) = 2T(n/2) + \Theta(1) \to \mathbf{\Theta(n)}$, санах ой $\Theta(n)$ (зангилаа) + $\Theta(\lg n)$ стек. Модны өндөр яг $\lceil \lg(n+1) \rceil$.

3. Longest Nice Substring (LC 1763) — хэсэгчлэх.

«Эсрэг хэлбэр нь мөрөнд огт байхгүй үсэг (муу үсэг) агуулсан дэд мөр хэзээ ч сайхан болохгүй. Иймд мөрийг **бүх** муу үсгээр нь нэг дор хувааж, хэсэг бүрийг дахин дуудалтаар бодоод хамгийн уртыг (тэнцүү бол эхнийхийг) авна. Муу үсэг байхгүй бол мөр өөрөө сайхан.»

```
# prj1/B190910014/python/src/p3_longest_nice_substring.py
class Solution:
    def longestNiceSubstring(self, s: str) -> str:
        if len(s) < 2:
            return ""
        present = set(s)
        best = ""
        start = 0
        split = False
        for i, ch in enumerate(s):
            if ch.swapcase() not in present:  # муу үсэг
                split = True
                part = self.longestNiceSubstring(s[start:i])
                if len(part) > len(best):  # тэнцүү бол эхнийхийг үлдээнэ
                    best = part
                start = i + 1
        if not split:
            return s  # муу үсэггүй - бүхэлдээ сайхан
        part = self.longestNiceSubstring(s[start:])
        return part if len(part) > len(best) else best
```

Нэг төвшинд хуваасан муу үсгийн төрөл хэсгүүдээс бүрмөсөн алга болдог тул гүн ≤ 52. Төвшин бүр $\Theta(n) \to$ нийт $\mathbf{\Theta(n)}$. Зөвхөн эхний муу үсгээр хуваавал «аааа…а» мэт оролтод гүн n болж $\Theta(n^2)$ болно.

4. Different Ways to Add Parentheses (LC 241).

Анхны (B төвшин): «Хамгийн сүүлд хийгдэх үйлдлийг сонгож хоёр хуваана. Хоёр талын бүх утгыг дахин дуудалтаар олоод, хос бүрт тэр үйлдлийг хэрэглэнэ.»

```
# prj1/B190910014/python/src/p4_different_ways.py
def apply(x: int, y: int, op: str) -> int:
    if op == "+":
        return x + y
    if op == "-":
        return x - y
    return x * y

class Solution:
    def __init__(self) -> None:
        self.calls = 0  # дахин дуудалтын тоо (хэмжилтэд)

    def diffWaysToCompute(self, expression: str) -> List[int]:
        self.calls += 1
        results = []
        for i, ch in enumerate(expression):
            if ch in "+-*":
                left = self.diffWaysToCompute(expression[:i])
                right = self.diffWaysToCompute(expression[i + 1:])
                for x in left:
                    for y in right:
                        results.append(apply(x, y, ch))
        if not results:  # үйлдэлгүй - ганц тоо
            results.append(int(expression))
        return results
```

Дуудалтын тоо $\mathbf{3^k}$ (k = үйлдлийн тоо), учир нь ижил дэд илэрхийллийг олон дахин боддог.

Сайжруулсан (A төвшин): «Илэрхийллийг нэг удаа тоо, үйлдлийн жагсаалт болгон задална. Дэд бодлогыг операндын товъёгийн муж (i, j)-ээр тодорхойлж, хариуг memo-д санана.»

```
# prj1/B190910014/python/src/p4_different_ways_improved.py
class Solution:
    def __init__(self) -> None:
        self.calls = 0  # memo-д байхгүй үед бодсон дэд бодлогын тоо (хэмжилтэд)

    def diffWaysToCompute(self, expression: str) -> List[int]:
        nums: list[int] = []
        ops: list[str] = []
        value = 0
        for ch in expression:
            if ch.isdigit():
                value = value * 10 + int(ch)
            else:
                nums.append(value)
                ops.append(ch)
                value = 0
        nums.append(value)

        memo: dict[tuple[int, int], list[int]] = {}

        def compute(i: int, j: int) -> list[int]:  # nums[i..j] операндууд
            if (i, j) in memo:
                return memo[(i, j)]
```

```
            self.calls += 1
            if i == j:
                result = [nums[i]]
            else:
                result = []
                for k in range(i, j):  # ops[k] нь nums[k], nums[k+1]-ийн хооронд
                    for x in compute(i, k):
                        for y in compute(k + 1, j):
                            result.append(apply(x, y, ops[k]))
            memo[(i, j)] = result
            return result

        return compute(0, len(nums) - 1)
```

Өөр дэд бодлого ердөө $\mathbf{(k+1)(k+2)/2}$ (k = 10 үед 59 049 → 66 дуудалт). Хариуны тоо нь Каталаны тоо C_k. Хугацаа Python-д 6.5 дахин, Java-д 8.8 дахин буурсан.

5. Construct Binary Tree from Preorder and Inorder (LC 105).

Анхны (B төвшин): «preorder[0] нь үндэс. Түүнийг inorder-оос хайж олоод, хоёр жагсаалтыг зүсэж хоёр дэд модыг дахин дуудалтаар байгуулна.»

```
# prj1/B190910014/python/src/p5_build_tree.py
class Solution:
    def buildTree(self, preorder: List[int], inorder: List[int]) -> Optional[TreeNode]:
        if not preorder:
            return None
        root = TreeNode(preorder[0])
        k = inorder.index(preorder[0])  # Θ(n) хайлт
        root.left = self.buildTree(preorder[1:k + 1], inorder[:k])
        root.right = self.buildTree(preorder[k + 1:], inorder[k + 1:])
        return root
```

Гинжин (нэг талдаа тулсан) модонд $T(n) = T(n-1) + \Theta(n) = \mathbf{\Theta(n^2)}$ хугацаа, санах ой.

Сайжруулсан (A төвшин): «inorder-ийн утга → байрлалыг толинд нэг удаа хадгална (хайлт Θ(1)). Жагсаалт хуулахгүй, inorder-ийн (lo, hi) мужаар дамжуулна. preorder-ийг нэг заагчаар дараалаар уншина.»

```
# prj1/B190910014/python/src/p5_build_tree_improved.py
class Solution:
    def buildTree(self, preorder: List[int], inorder: List[int]) -> Optional[TreeNode]:
        position = {value: i for i, value in enumerate(inorder)}
        next_root = 0  # preorder-оос дараагийн үндсийг уншина

        def build(lo: int, hi: int) -> Optional[TreeNode]:  # inorder[lo..hi]
            nonlocal next_root
            if lo > hi:
                return None
            value = preorder[next_root]
            next_root += 1
            k = position[value]
            root = TreeNode(value)
            root.left = build(lo, k - 1)  # preorder-т зүүн дэд мод түрүүлж ирдэг
            root.right = build(k + 1, hi)
            return root

        return build(0, len(inorder) - 1)
```

Модны хэлбэрээс үл хамааран $\Theta(\mathbf{n})$. n = 4000 гинжин модонд Python-д 43 дахин, Java-д 186 дахин хурдан.

8.3 Бие даалт 2 — Графын хайлт, мөрийн хайлт, хомхойлох (`prj2/B190910014`)

1. strStr (LC 28) — KMP, бэлэн хайлтын функцгүй.

«needle-ийн угтвар бүрийн хамгийн урт жинхэнэ угтвар = дагаврыг fail массивт урьдчилан тооцоолно. haystack-ийг нэг гүйхдээ тааралгүй болбол needle-ийн заагчийг fail-ээр ухраана, haystack-ийн заагчийг ухраахгүй.»

```
# prj2/B190910014/python/src/p1_str_str.py
class Solution:
    def strStr(self, haystack: str, needle: str) -> int:
        m = len(needle)
        if m == 0:
            return 0
        fail = [0] * m
        k = 0
        for i in range(1, m):
            while k > 0 and needle[i] != needle[k]:
                k = fail[k - 1]
            if needle[i] == needle[k]:
                k += 1
            fail[i] = k
        k = 0
        for i, ch in enumerate(haystack):
            while k > 0 and ch != needle[k]:
                k = fail[k - 1]
            if ch == needle[k]:
                k += 1
                if k == m:
                    return i - m + 1
        return -1
```

Гараар: needle = «abab» → fail = $[0, 0, 1, 2]$. $\Theta(\mathbf{n + m})$. Харьцуулах энгийн арга (`SolutionNaive`) байрлал бүрээс тэмдэгт тэмдэгтээр харьцуулдаг, O(n·m).

2. Flood Fill (LC 733) — BFS.

«Эхлэх нүднээс ижил өнгөтэй 4 хөрш уруу тархана. Нүдийг дараалалд оруулахдаа будна, тэгвэл дахин орохгүй. Шинэ өнгө хуучинтай ижил бол шууд буцаана, эс бөгөөс зогсохгүй.»

```
# prj2/B190910014/python/src/p2_flood_fill.py
class Solution:
    def floodFill(self, image: List[List[int]], sr: int, sc: int, color: int) -> List[List[int]]:
        old = image[sr][sc]
        if old == color:
            return image
        rows, cols = len(image), len(image[0])
        image[sr][sc] = color
        queue = deque([(sr, sc)])
        while queue:
            r, c = queue.popleft()
            for nr, nc in ((r - 1, c), (r + 1, c), (r, c - 1), (r, c + 1)):
                if 0 <= nr < rows and 0 <= nc < cols and image[nr][nc] == old:
                    image[nr][nc] = color
                    queue.append((nr, nc))
        return image
```

$\Theta(\mathbf{R \cdot C})$ — нүд бүр нэг удаа. Мөн файлд стектэй DFS хувилбар (`SolutionDfs`) бий.

3. Assign Cookies (LC 455) — хомхойлох.

«Хоёуланг эрэмбэлнэ. Жигнэмгийг жижгээс нь авч, одоогийн хамгийн бага шуналтай хүүхдийг хангаж чадвал түүнд өгнө, эс бөгөөс энэ жигнэмэг хэнд ч хүрэхгүй.»

```
# prj2/B190910014/python/src/p3_assign_cookies.py
class Solution:
    def findContentChildren(self, g: List[int], s: List[int]) -> int:
        g.sort()
        s.sort()
        child = 0
        for cookie in s:
            if child < len(g) and cookie >= g[child]:
                child += 1
        return child
```

$\Theta(\mathbf{n \lg n + m \lg m})$. Зөв гэдгийг солих аргаар: хамгийн бага шуналтай хүүхдэд хангах хамгийн жижиг жигнэмгийг өгөх нь алдагдалгүй.

4. Jump Game II (LC 45).

Анхны (B төвшин) — динамик программчлал: «dp[i] = 0-ээс i-д хүрэх хамгийн цөөн үсрэлт. i бүрээс хүрч болох j-үүдийг dp[i] + 1-ээр шинэчилнэ.»

```
# prj2/B190910014/python/src/p4_jump_game.py
class Solution:
    def __init__(self) -> None:
        self.steps = 0  # dp-г шинэчилсэн тоо (хэмжилтэд)

    def jump(self, nums: List[int]) -> int:
        n = len(nums)
        inf = n
        dp = [inf] * n
        dp[0] = 0
        for i in range(n - 1):
            if dp[i] == inf:
                continue
            nxt = dp[i] + 1
            for j in range(i + 1, min(n, i + nums[i] + 1)):
                self.steps += 1
                if nxt < dp[j]:
                    dp[j] = nxt
        return dp[-1]
```

$O(n \cdot \max \text{nums}) \leq \Theta(\mathbf{n^2})$, санах ой $\Theta(n)$.

Сайжруулсан (A төвшин) — хомхойлох = далд BFS: «k үсрэлтээр хүрэх байрлалууд нь тасралтгүй муж. Тухайн мужаас хамгийн хол хүрэх цэгийг тооцоолж явна. Муж дуусмагц нэг үсрэлт нэмж, шинэ муж нь хамгийн хол цэг хүртэл.»

```
# prj2/B190910014/python/src/p4_jump_game_improved.py
class Solution:
    def jump(self, nums: List[int]) -> int:
        jumps = 0
        current_end = 0  # одоогийн төвшний хамгийн хол байрлал
        farthest = 0     # дараагийн төвшний хамгийн хол байрлал
```

```
        for i in range(len(nums) - 1):
            if i + nums[i] > farthest:
                farthest = i + nums[i]
            if i == current_end:  # энэ төвшин дууслаа - нэг үсрэлт нэмнэ
                jumps += 1
                current_end = farthest
                if current_end >= len(nums) - 1:
                    break
        return jumps
```

Θ(n), Θ(1). Гараар [2, 3, 1, 1, 4]: i = 0 → farthest 2, i = current_end → jumps 1, current_end 2; i = 1 → farthest 4; i = 2 = current_end → jumps 2, current_end 4 ≥ 4 → зогсоно. Хариу 2. Python-д n = 10^4 үед ДП-ээс 755 дахин хурдан.

5. Open the Lock (LC 752).

Анхны (B төвшин) — BFS: «Төлөв бүр 4 оронтой мөр, нэг алхмаар 8 хөрш (орон бүрийг ±1). Жингүй граф дээр хамгийн богино зам → BFS. Түгжих хослол уруу орохгүй. Шинэ төлөвийг түгжээ эсэхийг шалгасны **дараа** зорилготой харьцуулна.»

```
# prj2/B190910014/python/src/p5_open_lock.py
class Solution:
    def openLock(self, deadends: List[str], target: str) -> int:
        dead = set(deadends)
        if "0000" in dead:
            return -1
        if target == "0000":
            return 0
        seen = {"0000"}
        queue = deque([("0000", 0)])
        while queue:
            state, dist = queue.popleft()
            for i in range(4):
                digit = int(state[i])
                for d in (-1, 1):
                    nxt = state[:i] + str((digit + d) % 10) + state[i + 1:]
                    if nxt in seen or nxt in dead:  # зорилго өөрөө түгжих хослол бол хүрэхгүй
                        continue
                    if nxt == target:
                        return dist + 1
                    seen.add(nxt)
                    queue.append((nxt, dist + 1))
        return -1
```

(Файлд мөн тэлсэн төлөвийг тоолох `self.expanded` тоолуур бий.) **Анхаар:** эхний хувилбарт зорилгыг түгжээнээс өмнө шалгадаг байсан тул зорилго өөрөө түгжих хослол үед буруу хариу өгч байсан. Санамсаргүй тестээр илрүүлж зассан.

Сайжруулсан (A төвшин) — хоёр талын BFS, төлөв нь 0…9999 тоо: «Хөршийг 10-ын зэргийг нэмж/хасаж олно. "0000"ба зорилгоос ээлжлэн, жижиг хилийг нь нэг төвшин тэлнэ. Хоёр тал уулзмагц хариу нь хоёр талын төвшний нийлбэр.»

```
# prj2/B190910014/python/src/p5_open_lock_improved.py
POW = (1000, 100, 10, 1)

def neighbours(x: int):
    for p in POW:
        digit = x // p % 10
```

```
        yield x + p if digit < 9 else x - 9 * p  # +1 (9 → 0)
        yield x - p if digit > 0 else x + 9 * p  # −1 (0 → 9)

class Solution:
    def openLock(self, deadends: List[str], target: str) -> int:
        dead = bytearray(10000)
        for d in deadends:
            dead[int(d)] = 1
        start, goal = 0, int(target)
        if dead[start] or dead[goal]:
            return -1
        if start == goal:
            return 0
        side = bytearray(10000)  # 0 - үзээгүй, 1 - эхлэлээс, 2 - зорилгоос хүрсэн
        side[start], side[goal] = 1, 2
        front = {1: [start], 2: [goal]}
        depth = {1: 0, 2: 0}
        while front[1] and front[2]:
            s = 1 if len(front[1]) <= len(front[2]) else 2  # жижиг хилийг тэлнэ
            other = 3 - s
            depth[s] += 1
            nxt = []
            for x in front[s]:
                for y in neighbours(x):
                    if dead[y] or side[y] == s:
                        continue
                    if side[y] == other:
                        return depth[1] + depth[2]
                    side[y] = s
                    nxt.append(y)
            front[s] = nxt
        return -1
```

Нээх төлөв $\approx 2 \cdot b^{d/2}$ (энгийн BFS-д b^d). LeetCode-ийн жишээнд тэлсэн төлөв 417 → 46. Зорилго бүслэгдсэн үед энгийн BFS бүх 9 991 төлөвийг нээдэг бол энэ хувилбар 2 төлөв нээгээд л −1 гэж мэднэ.

8.4 Хамгаалалтын өмнөх шалгах жагсаалт

- Бодлого бүрийг хоосон редакторт **өөрөө** бичиж, тестээ ажиллуулж үзэх: `cd prj1/B190910014/ python && .venv/bin/pytest -v` (prj2-т мөн адил).
- Бодлого бүрийн хугацаа, санах ойг **яагаад** гэдэгтэй нь хэлж чаддаг байх.
- Сайжруулсан хувилбар (4, 5-р бодлого) өмнөхөөсөө юугаараа сайн болохыг тоогоор хэлэх (жишээ нь «755 дахин хурдан»).
- Тайлан дахь улаанаар зурагдах гадаад үгсийн утгыг хэлж чаддаг байх: inorder (зүүн → үндэс → баруун тойролт), preorder (үндэс → зүүн → баруун), memo (санах), BFS (гүехэн хайлт), КМР (Кнут–Моррис–Праттын арга), JIT (Java-гийн ажиллах үеийн хөрвүүлэгч).
- Java хувилбарыг ч мэддэг байх: ижил санаа, файлууд `java/src/main/java/com/prj1/`, `com/ prj2/`.

9 Нэгдсэн зураг — бүх зүйл хэрхэн холбогдох вэ

Лаб	Сэдэв	Гол санаа	Дараагийн лабтай холбоо
1	Орчин, Фибоначчи	шууд дахин дуудалт давхардсан ажил хийж экспоненциал болж болно	Лаб 4-ийн ДП үүнийг хүснэгтээр засна
2	Хугацааг тоолох	алхмыг тоолж томьёо гаргах, давталтын мод	Лаб 3-ын ерөнхий арга модыг товчилдог
3	Хэсэгчлэх	$T(n) = aT(n/b) + f(n)$, ерөнхий арга, Штрассен	дэд бодлого давхцвал → Лаб 4
4	Динамик программчлал	дэд бодлогыг нэг удаа бодож хадгалах	ганц сонголт хангалттай бол → Лаб 5
5	Хомхойлох	алхам бүрд хамгийн сайн сонголт, солих аргаар батлах	мөнгөний бодлого: хомхойлох (каноник) ба ДП (ерөнхий)

Нэг бодлого — гурван арга (мөнгө хариулах):

- **Бүх боломжийг шалгах** (дахин дуудалт): экспоненциал.
- **ДП** (min_coins): $\Theta(m \cdot x)$, ямар ч системд зөв.
- **Хомхойлох** (make_change): $\Theta(m)$, зөвхөн каноник системд зөв.

Нэг бодлого — гурван арга (Фибоначчи): шууд рекурс $\Theta(\varphi^n)$ → давталт (ДП) $\Theta(n)$ → матрицын зэрэг (хэсэгчлэх) $\Theta(\lg n)$.

Сайжруулалтын ерөнхий зарчмууд (А төвшинд хэрэглэсэн):

1. **Давхардлыг арилгах:** memo, хүснэгт (Бие даалт 1-ийн 4-р бодлого, Лаб 4).
2. **Хайлтыг хурдасгах:** толь, урьдчилан тооцоолсон массив (Бие даалт 1-ийн 5-р, Лаб 5-ын next_use).
3. **Хуулахгүй байх:** товъёгоор дамжуулах (Лаб 3-ын дахин дуудах матриц, Бие даалт 1).
4. **Бүтцийн ажиглалт:** тасралтгүй муж (Jump Game II), каноник систем (мөнгө).
5. **Жижиг оролтод энгийн арга:** Штрассены cutoff.
6. **Хэмжиж шалгах:** онол хангалтгүй — тогтмол үржүүлэгч, кэш, санах ойн хуваарилалт бодит хугацаанд нөлөөлдөг.

10 Түгээмэл асуултын хурдан хариулт

Асуулт	Товч хариулт
Θ, O, Ω-ийн ялгаа?	Θ — яг, O — дээд хязгаар, Ω — доод хязгаар
Яагаад тогтмолыг орхидог вэ?	n томрох үед өсөлтийн хурд л чухал
Python яагаад Java-гаас удаан бэ?	тайлбарлагч хэл; Java JIT-ээр машины код болгодог. Эрэмбэ ижил, тогтмол 10–300 дахин
Хэмжилтийг яагаад олон удаа хийдэг вэ?	компьютерийн ачаалал, JIT, GC-ийн хэлбэлзлийг багасгах; медиан авна
Тестийг яаж бичсэн бэ?	багшийн TSV + санамсаргүй оролтыг brute force-той тулгах
Санамсаргүй тест юунд хэрэгтэй вэ?	гараар бодоогүй тохиолдлын алдааг илрүүлнэ (Open the Lock-ийн алдааг ингэж олсон)
Оргил ба нийт хуваарилалтын ялгаа?	оргил — нэгэн зэрэг эзэлсэн хамгийн их; нийт — бүх хуваарилалтын нийлбэр
Рекурсийн стек хэр их санах ой эзлэх вэ?	гүнтэй пропорциональ: хоёртын хайлт Θ(lg n), гинжин мод Θ(n)
Ерөнхий арга хэзээ хэрэглэгдэхгүй вэ?	f нь зааг дээр (R6), эсвэл хуваалт тэгш биш (T(k) + T(n−1−k))
ДП-д юу хадгалах вэ?	дэд бодлогын хариу; шийд сэргээх бол сонголтыг (s массив)
Хомхойлох арга зөв гэдгийг яаж батлах вэ?	солих арга: оновчтой шийдийн нэг сонголтыг хомхойлохоор солиход муудахгүй

Амжилт хүсье! Хамгаалалтын өмнө энэ баримтыг дахин уншиж, код бүрийг өөрөө нээж хараарай. Ойлгохгүй хэсэг байвал асуугаарай.